形式语言与自动机
Formal Languages and Automata Theory

有穷状态自动机

计算机科学与技术学院
哈尔滨工业大学（深圳）

郑宜峰

主要内容

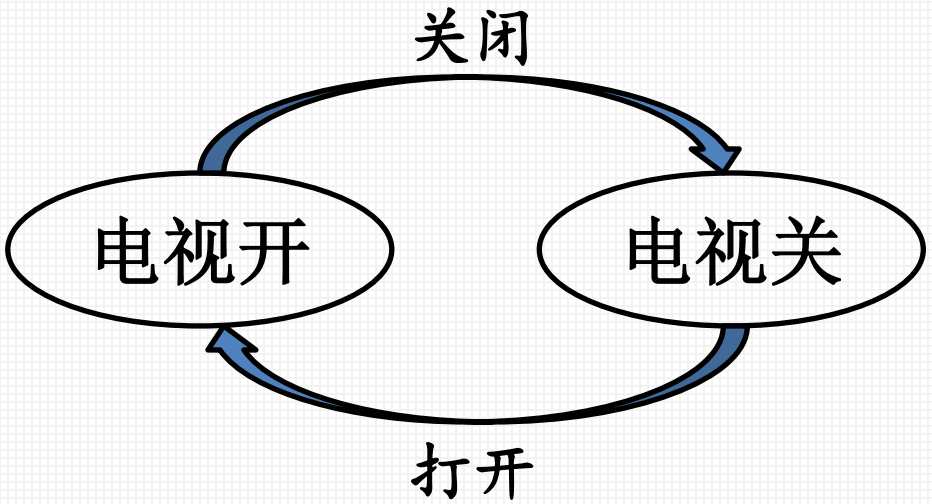
- 确定的有穷自动机
- 非确定有穷自动机
- 带空转移的非确定有穷自动机

什么是有穷自动机

- 有穷自动机分为确定型的有穷自动机（DFA）与非确定型的有穷自动机（NFA）两种。

有穷状态系统

- 在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行状态的迁移。
- 例:电视遥控器开关。

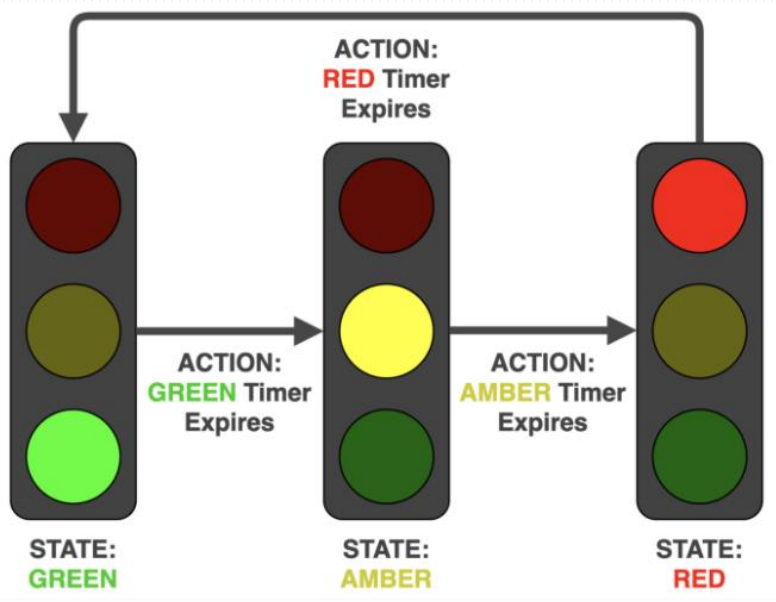


状态

输入字符/命令—关闭/打开

有穷状态系统

- 在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行状态的迁移。
- 例:交通信号灯。

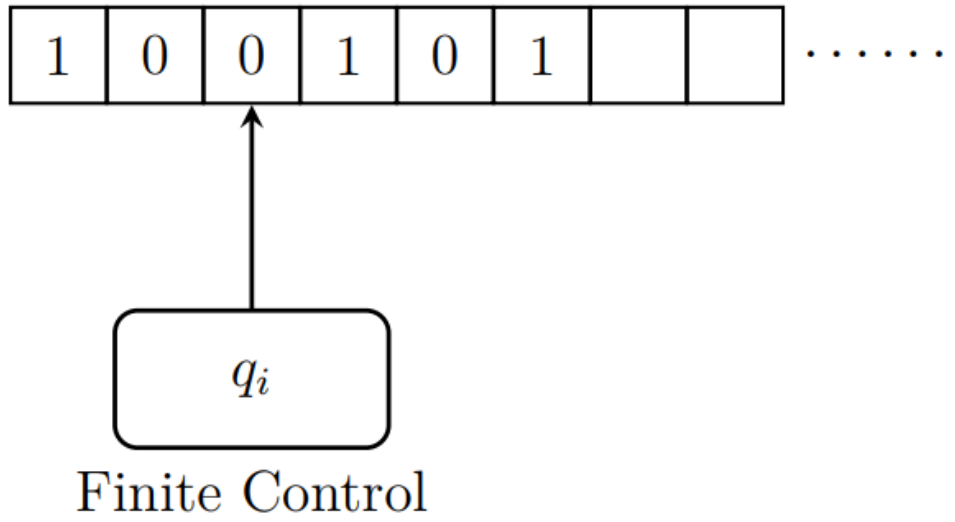


有穷状态系统

- 在系统的运行过程中，系统的状态在有限状态间不断变化，每个状态可以迁移到零个或多个状态，系统的输入决定执行状态的迁移。

有穷自动机抽象装置

- 一条输入带
- 一个读头
- 一个有穷控制器



- 输入带划分为单元格，每个格子可以放置一个字符，整条输入带可以表示一个符号串。
- 读头可以从左到右扫描输入带，将每次的字符传输给控制器。
- 控制器根据读入的字符和当前的状态，进行状态转移。然后控制读头进行下一个字符的读取。

形式化定义

•定义：确定的有穷自动机(DFA, Deterministic Finite Automaton)A为五元组$(Q, \Sigma, \delta, q_0, F)$

(1) Q:有穷状态集；

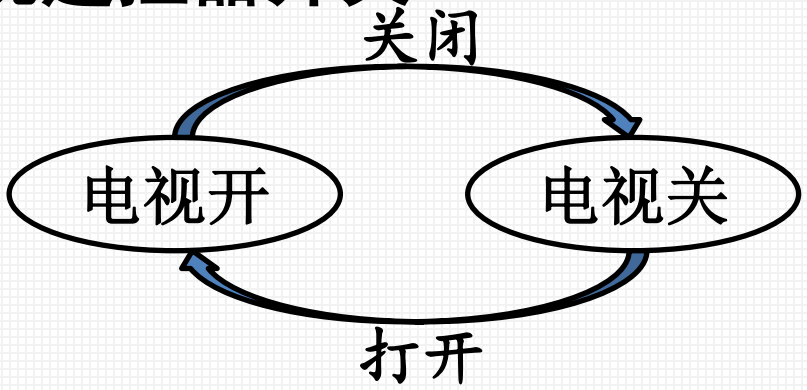
(2) Σ:有穷输入符号集或字母表；(a finite set of input symbols/alphabet)

(3) $\delta: Q \times \Sigma \longrightarrow Q$:状态转移函数(a transition function)。刻画DFA动作

(4) $q_0 \in Q$: 初始状态(a start state)；

(5) $F \subseteq Q$: 终结状态集或接受状态集(a set of accepting states)。

- **例：电视遥控器开关**



状态

关闭/打开—输入字符/命令

  - **字母表Σ={ *p* }（*p* — 按开关键）**
  - **有穷状态集*Q*：**

    (1) 电视关（q_0）

    (2) 电视开（q_1）
  - **状态转换函数δ：**

    $\delta(q_0, p)=q_1$，$\delta(q_1, p)=q_0$

确定的有穷自动机示例

- **例：设计DFA，在任何由0和1构成的串中，接受含有01子串的全部串。**
  - 字母表$\Sigma = \{0,1\}$

确定的有穷自动机示例

- **例：设计DFA，在任何由0和1构成的串中，接受含有01子串的全部串。**
  - 字母表$\Sigma = \{0,1\}$
  - 有穷状态集Q：
    - 没发现01子串，且0也还没出现（q_0-初始状态）

确定的有穷自动机示例

- **例：设计DFA，在任何由0和1构成的串中，接受含有01子串的全部串。**
  - 字母表$\Sigma = \{0,1\}$
  - 有穷状态集Q：
    - 没发现01子串，且0也还没出现（q_0-初始状态）
    - 没发现01子串，但刚刚已经读入了一个0，后面只需再读入一个1就符合条件了(q_1)

确定的有穷自动机示例

- **例：设计DFA，在任何由0和1构成的串中，接受含有01子串的全部串。**
  - 字母表$\Sigma = \{0,1\}$
  - 有穷状态集Q：
    - 没发现01子串，且0也还没出现（q_0-初始状态）
    - 没发现01子串，但刚刚已经读入了一个0，后面只需再读入一个1就符合条件了(q_1)
    - 已经发现01子串，不再关心串的其余部分（q_2-终止）

确定的有穷自动机示例

- **例：设计DFA，在任何由0和1构成的串中，接受含有01子串的全部串。**
  - 字母表$\Sigma = \{0,1\}$
  - 有穷状态集Q:
    - 没发现01子串，且0也还没出现（q_0-初始状态）
    - 没发现01子串，但刚刚已经读入了一个0，后面只需再读入一个1就符合条件了(q_1)
    - 已经发现01子串，不再关心串的其余部分（q_2-终止）
  - 状态转换函数:

$$\delta(q_0,0) = q_1,\ \delta(q_1,0) = q_1,\ \delta(q_2,0) = q_2$$
$$\delta(q_0,1) = q_0,\ \delta(q_1,1) = q_2,\ \delta(q_2,1) = q_2$$

状态转移图和状态转移表

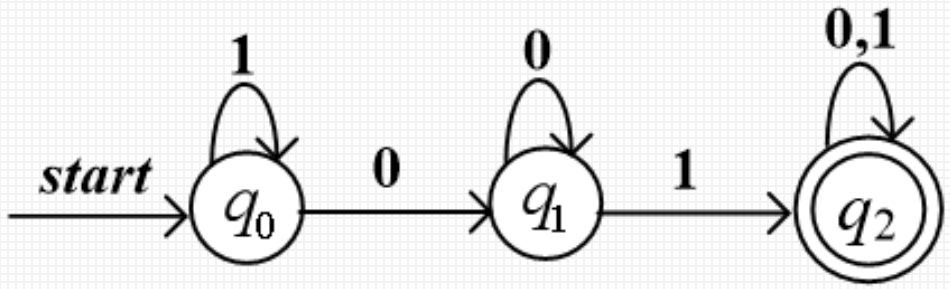
- **状态转移有两种直观的表示方式**

状态转移图

状态转移表

状态转移图

$$\delta(q_0,0)=q_1,\ \delta(q_1,0)=q_1,\ \delta(q_2,0)=q_2$$
$$\delta(q_0,1)=q_0,\ \delta(q_1,1)=q_2,\ \delta(q_2,1)=q_2$$



- 每个状态q对应一个节点，用圆圈表示；
- 状态转移$\delta(q,a)=p$为一条从q到p且标记为输入字符a的有向边；
- 开始状态q_0用一个标有start的箭头表示；
- 接受状态的节点，用双圆圈表示。

状态转移表

$$\delta(q_0,0)=q_1,\ \delta(q_1,0)=q_1,\ \delta(q_2,0)=q_2$$
$$\delta(q_0,1)=q_0,\ \delta(q_1,1)=q_2,\ \delta(q_2,1)=q_2$$

	0	1
→q_0	q_1	q_0
q_1	q_1	q_2
*q_2	q_2	q_2

- 每个状态q对应一行，每个输入字符对应一列；
- 若有$\delta(q,a)=p$，用第q行第a列中填入的p表示；
- 开始状态q_0前，标记箭头→表示；
- 接受状态$q \in F$前，标记星号*表示。

典型问题

- 设计一个DFA，使其接受且仅接受给定的语言L（符号串的集合）。

DFA的设计

- 例:若$\Sigma = \{0,1\}$,给出接受全部含有奇数个1的串的DFA。

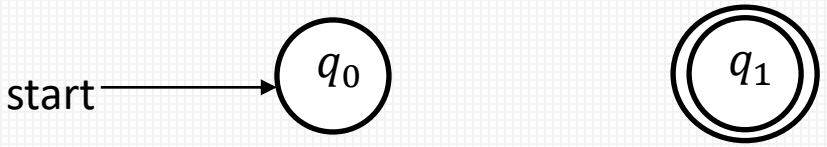
- **例:若$\Sigma = \{0,1\}$, 给出接受全部含有奇数个1的串的DFA。**
  - 状态转移函数

    δ(even,0) = even　　δ(even,1) = odd

    δ(odd,0) = odd　　δ(odd,1) = even
  - 状态转移表

状态\输入	0	1
even	even	odd
odd	odd	even

  - 状态转移图



表示有偶数个1　　表示有奇数个1

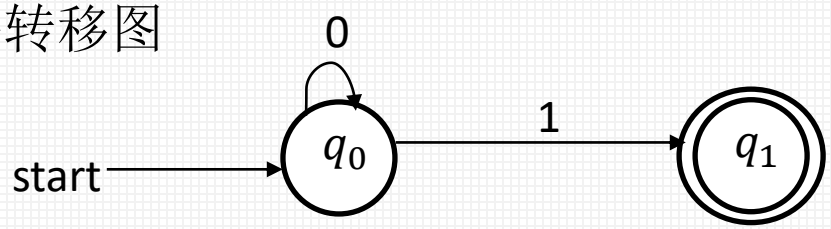
- **例:若$\Sigma = \{0,1\}$, 给出接受全部含有奇数个1的串的DFA。**
  - 状态转移函数

    δ(even,0) = even　　δ(even,1) = odd

    δ(odd,0) = odd　　δ(odd,1) = even
  - 状态转移表

状态\输入	0	1
even	even	odd
odd	odd	even

  - 状态转移图



表示有偶数个1　　表示有奇数个1

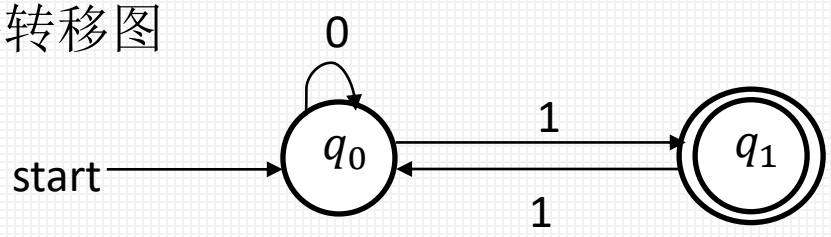
- **例:若$\Sigma = \{0,1\}$, 给出接受全部含有奇数个1的串的DFA。**
  - 状态转移函数

    δ(even,0) = even　　δ(even,1) = odd

    δ(odd,0) = odd　　δ(odd,1) = even
  - 状态转移表

状态\输入	0	1
even	even	odd
odd	odd	even

  - 状态转移图



表示有偶数个1　　表示有奇数个1

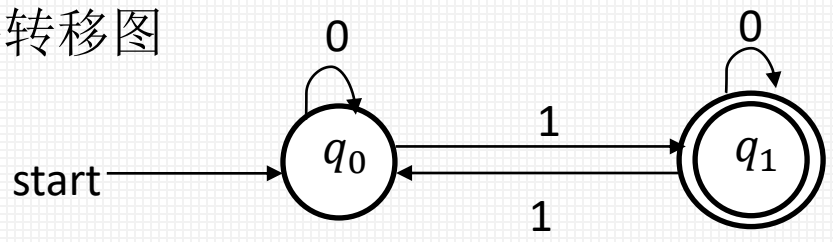
- **例:若$\Sigma = \{0,1\}$, 给出接受全部含有奇数个1的串的DFA。**
  - 状态转移函数

    $\delta(even,0) = even$　　$\delta(even,1) = odd$

    $\delta(odd,0) = odd$　　$\delta(odd,1) = even$
  - 状态转移表

状态\输入	0	1
even	even	odd
odd	odd	even

  - 状态转移图

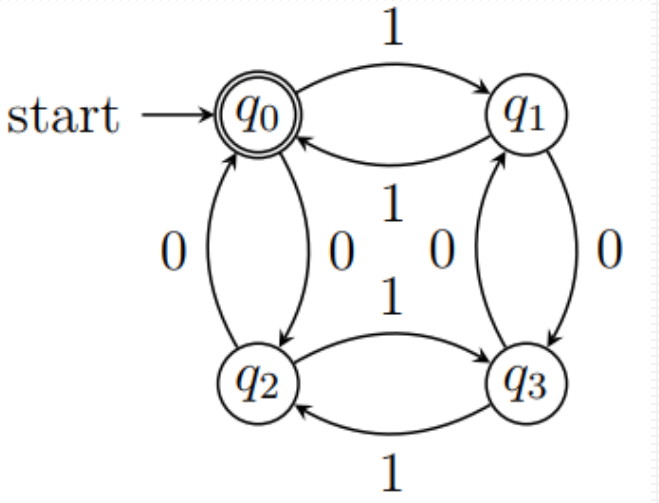


表示有偶数个1　　表示有奇数个1

DFA的设计

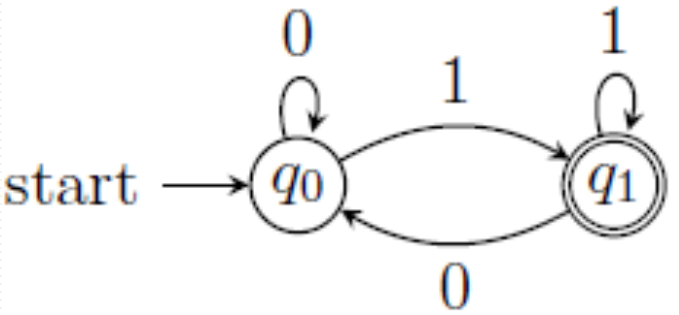
- 例:若$\Sigma = \{0,1\}$,给出接受全部含有偶数个0和偶数个1的串DFA。

- **例:若$\Sigma = \{0,1\}$,给出接受全部含有偶数个0和偶数个1的串DFA。**
- 四个状态: q_0:(偶0，偶1)，q_1:(偶0,奇1)，
  q_2:(奇0，偶1)，q_3:(奇0,奇1)。

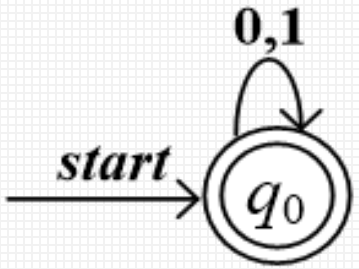


- **例：**

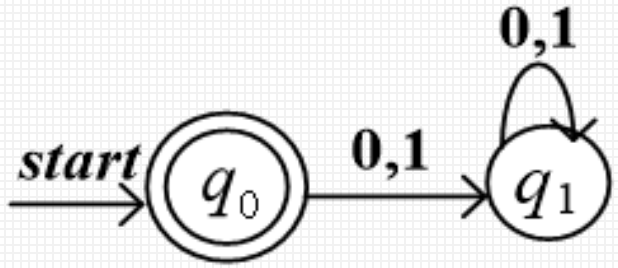
Σ={0,1}, 给出接受全部以1 结尾的串的DFA



Σ={0,1}, 给出接受Σ*的DFA



Σ={0,1}, 给出接受{ε}的DFA



扩展状态转移函数

- **DFA实际读入的是一个字符串，而状态转移函数处理的是一个字符，为了理论分析的时候更加便表示对字符串的处理，定义扩展状态转移函数。**

- **将状态转移函数δ扩展到对字符串的处理，定义扩展状态转移函数$\hat{\delta}$(delta-hat)：**

$$\hat{\delta}(q,w)=\begin{cases} q, & w=\varepsilon \\ \delta\left(\hat{\delta}(q,x),a\right), & w=xa \end{cases}$$

其中，a是一个字符，w和x是字符串。

这是一个递归定义，对每个串w，把它最后的一个字母（设为a）取出，分解为$w=xa$。

扩展状态转移函数

- **将状态转移函数δ扩展到对字符串的处理，定义扩展状态转移函数$\hat{\delta}$(delta-hat)：**

$$\hat{\delta}(q,w)=\begin{cases} q, & w=\epsilon \\ \delta\left(\hat{\delta}(q,x),a\right), & w=xa \end{cases}$$

其中，a是一个字符，w和x是字符串。

那么，当$w=a_0a_1a_2\ldots a_n$,则有

$$\hat{\delta}(q,w)=\delta(\hat{\delta}(q,a_0a_1\cdots a_{n-1}),a_n)$$

扩展状态转移函数

- **将状态转移函数δ扩展到对字符串的处理，定义扩展状态转移函数$\hat{\delta}$(delta-hat)：**

$$\hat{\delta}(q,w)=\begin{cases} q, & w=\epsilon \\ \delta\left(\hat{\delta}(q,x),a\right), & w=xa \end{cases}$$

其中，a是一个字符，w和x是字符串。

那么，当$w=a_0a_1a_2\ldots a_n$,则有

$$\hat{\delta}(q,w)=\delta(\hat{\delta}(q,a_0a_1\cdots a_{n-1}),a_n)$$

$$=\delta(\delta(\hat{\delta}(q,a_0a_1\cdots a_{n-2}),a_{n-1}),a_n)$$

扩展状态转移函数

- **将状态转移函数δ扩展到对字符串的处理，定义扩展状态转移函数$\hat{\delta}$(delta-hat)：**

$$\hat{\delta}(q,w)=\begin{cases} q, & w=\epsilon \\ \delta\big(\hat{\delta}(q,x),a\big), & w=xa \end{cases}$$

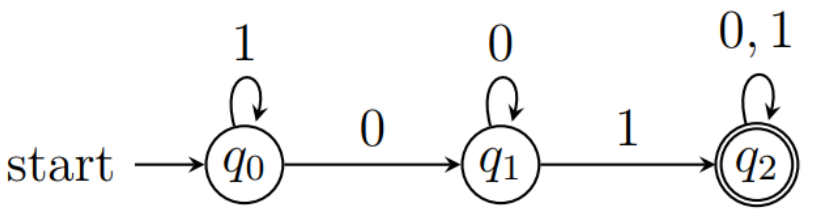
其中，a是一个字符，w和x是字符串。

那么，当$w=a_0a_1a_2\ldots a_n$,则有

$$\begin{aligned}\hat{\delta}(q,w)&=\delta(\hat{\delta}(q,a_0a_1\cdots a_{n-1}),a_n)\\&=\delta(\delta(\hat{\delta}(q,a_0a_1\cdots a_{n-2}),a_{n-1}),a_n)\\&=\cdots\\&=\delta(\delta(\cdots\delta(\hat{\delta}(q,\varepsilon),a_0)\cdots),a_{n-1}),a_n)\end{aligned}$$

扩展状态转移函数使用示例

- 例：接受全部含有01子串的DFA，$\hat{\delta}$处理串 0101 的过程。



$$\hat{\delta}(q_0,0101)=\delta(\hat{\delta}(q_0,010),1)$$
$$=\delta(\delta(\hat{\delta}(q_0,01),0),1)$$
$$=\delta(\delta(\delta(\hat{\delta}(q_0,0),1),0),1)$$
$$=\delta(\delta(\delta(\delta(\hat{\delta}(q_0,\varepsilon),0),1),0),1)$$
$$=\delta(\delta(\delta(\delta(q_0,0),1),0),1)$$
$$=\delta(\delta(\delta(q_1,1),0),1)$$
$$=\delta(\delta(q_2,0),1)=\delta(q_2,1)=q_2$$

先由外到内

后由内到外

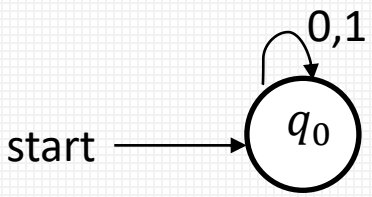
DFA的语言与正则语言

- **定义：若$D = (Q, \Sigma, \delta, q_0, F)$是一个DFA，则$D$识别的语言为**

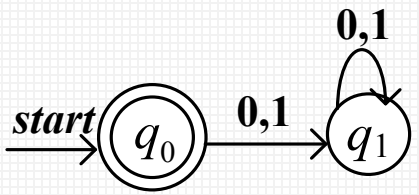
$$\mathbf{L}(D) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

即所有能使DFA从开始状态到达接受状态的符号串集合。

- **定义：如果语言L是某个DFA D识别的语言，则称其为正则语言。**
  - $\emptyset$，$\{\varepsilon\}$都是正则语言



∅的DFA: 无接受状态，即该DFA不接受任何符号串（包括空串)。



识别{ ε }的DFA。

主要内容

- 确定的有穷自动机
- **非确定有穷自动机**
- 带空转移的非确定有穷自动机

非确定性有限状态自动机理论的开创者

1976 年图灵奖获得者：
米凯尔. 拉宾和达纳. 斯科特

——非确定性有限状态自动机理论的开创者

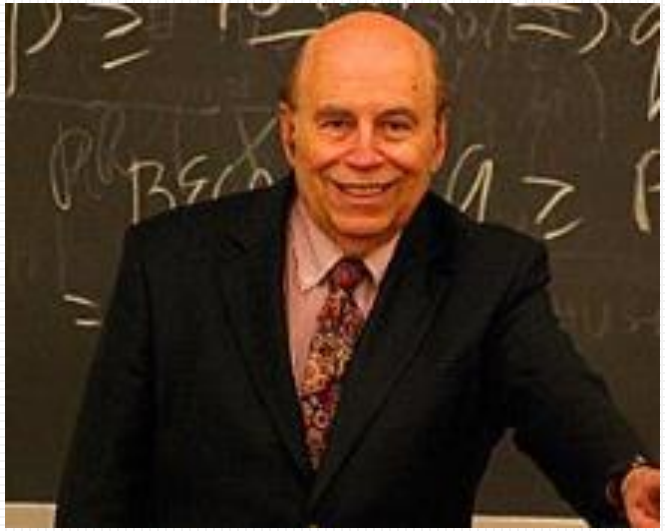
米凯尔·拉宾
Michael O. Rabin

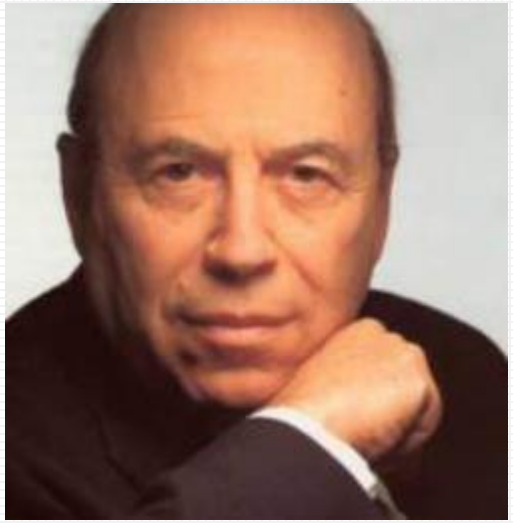
达纳·斯科特
Dana Steward Scott

1976 年度的图灵奖由当时在以色列希伯莱大学任教的米凯尔.拉宾和在英国牛津大学任数理逻辑教授的达纳.斯科特共同获得。

拉宾和斯科特是师兄弟，两人在20世纪50年代先后师从著名的逻辑学家和计算机专家阿隆索.邱奇（Alonzo Church,他因与Curry一起发明了λ-演算以及提出了“邱奇论题”的命题而闻名于世），并在有限自动机及其判定问题的研究中进行合作，奠定了非确定性有限状态自动机（NFA）的理论基础。

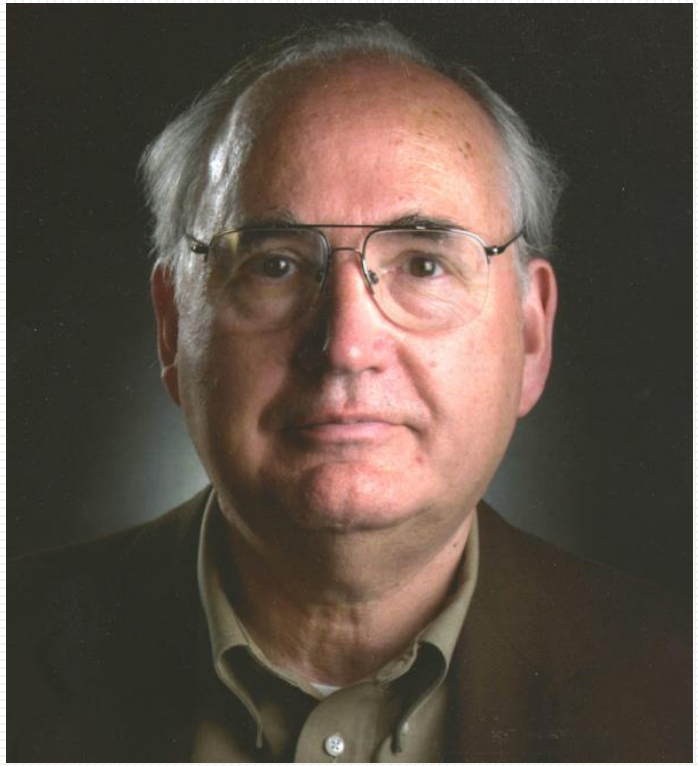
米凯尔·拉宾





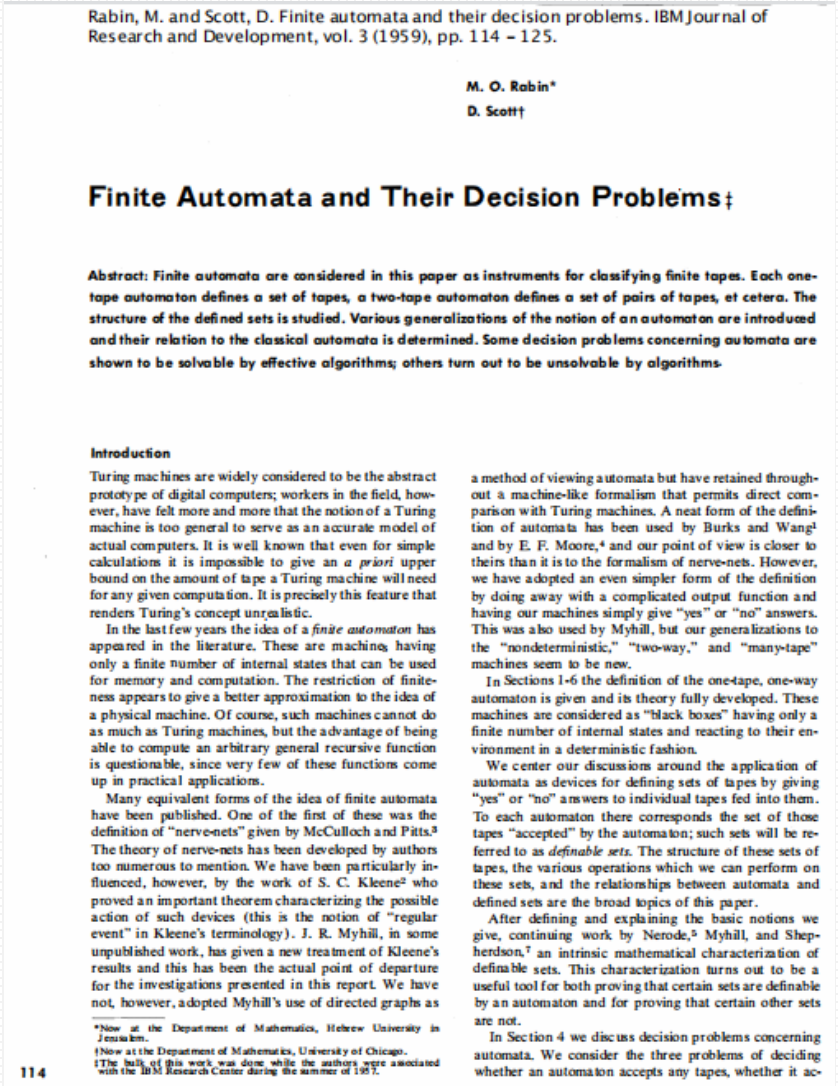
- 拉宾1931年9月1日生于德国的布雷斯劳。他父亲是一名犹太教教士，也是一位博士；拉宾的母亲也是知识分子，有文学博士头街。
- 从莱利学院（Reali College）毕业以后，拉宾进入希伯莱大学学习数学，在那里，他通过数学家克林（S. C. Kleene,因提出不动点定理而闻名于世）所著的《元数学》一书，首次接触到图灵关于可计算机的概念和图灵机这一理论计算模型，立即被深深吸引。
- 1953年，他获得希伯来大学的理学硕士，1956年获普林斯顿大学博士学位。

达纳·斯科特



- 斯科特1932年10月11日生于美国加利福尼亚州，在加州大学伯克利分校获得学士学位以后，进入普林斯顿大学研究生院学习，与米凯尔·拉宾一起师从阿隆索·邱奇，1958年取得博士学位。
- 他先后在芝加哥大学、加州大学伯克利分校、斯坦福大学、荷兰的阿姆斯特丹大学、普林斯顿大学和英国牛津大学等学府任教。1981年被卡内基梅隆大学聘为计算机科学、数理逻辑和哲学教授。

NFA的诞生背景



Rabin, M. and Scott, D. Finite automata and their decision problems. IBM Journal of Research and Development, vol. 3 (1959), pp. 114 – 125.

M. O. Rabin*
D. Scott†

Finite Automata and Their Decision Problems‡

Abstract: Finite automata are considered in this paper as instruments for classifying finite tapes. Each one-tape automaton defines a set of tapes, a two-tape automaton defines a set of pairs of tapes, et cetera. The structure of the defined sets is studied. Various generalizations of the notion of an automaton are introduced and their relation to the classical automata is determined. Some decision problems concerning automata are shown to be solvable by effective algorithms; others turn out to be unsolvable by algorithms.

Introduction

Turing machines are widely considered to be the abstract prototype of digital computers; workers in the field, however, have felt more and more that the notion of a Turing machine is too general to serve as an accurate model of actual computers. It is well known that even for simple calculations it is impossible to give an *a priori* upper bound on the amount of tape a Turing machine will need for any given computation. It is precisely this feature that renders Turing's concept unrealistic.

In the last few years the idea of a *finite automaton* has appeared in the literature. These are machines having only a finite number of internal states that can be used for memory and computation. The restriction of finiteness appears to give a better approximation to the idea of a physical machine. Of course, such machines cannot do as much as Turing machines, but the advantage of being able to compute an arbitrary general recursive function is questionable, since very few of these functions come up in practical applications.

Many equivalent forms of the idea of finite automata have been published. One of the first of these was the definition of "nerve-nets" given by McCulloch and Pitts.[3] The theory of nerve-nets has been developed by authors too numerous to mention. We have been particularly influenced, however, by the work of S. C. Kleene[2] who proved an important theorem characterizing the possible action of such devices (this is the notion of "regular event" in Kleene's terminology). J. R. Myhill, in some unpublished work, has given a new treatment of Kleene's results and this has been the actual point of departure for the investigations presented in this report. We have not, however, adopted Myhill's use of directed graphs as a method of viewing automata but have retained throughout a machine-like formalism that permits direct comparison with Turing machines. A neat form of the definition of automata has been used by Burks and Wang[1] and by E. F. Moore,[4] and our point of view is closer to theirs than it is to the formalism of nerve-nets. However, we have adopted an even simpler form of the definition by doing away with a complicated output function and having our machines simply give "yes" or "no" answers. This was also used by Myhill, but our generalizations to the "nondeterministic," "two-way," and "many-tape" machines seem to be new.

In Sections 1-6 the definition of the one-tape, one-way automaton is given and its theory fully developed. These machines are considered as "black boxes" having only a finite number of internal states and reacting to their environment in a deterministic fashion.

We center our discussions around the application of automata as devices for defining sets of tapes by giving "yes" or "no" answers to individual tapes fed into them. To each automaton there corresponds the set of those tapes "accepted" by the automaton; such sets will be referred to as *definable sets*. The structure of these sets of tapes, the various operations which we can perform on these sets, and the relationships between automata and defined sets are the broad topics of this paper.

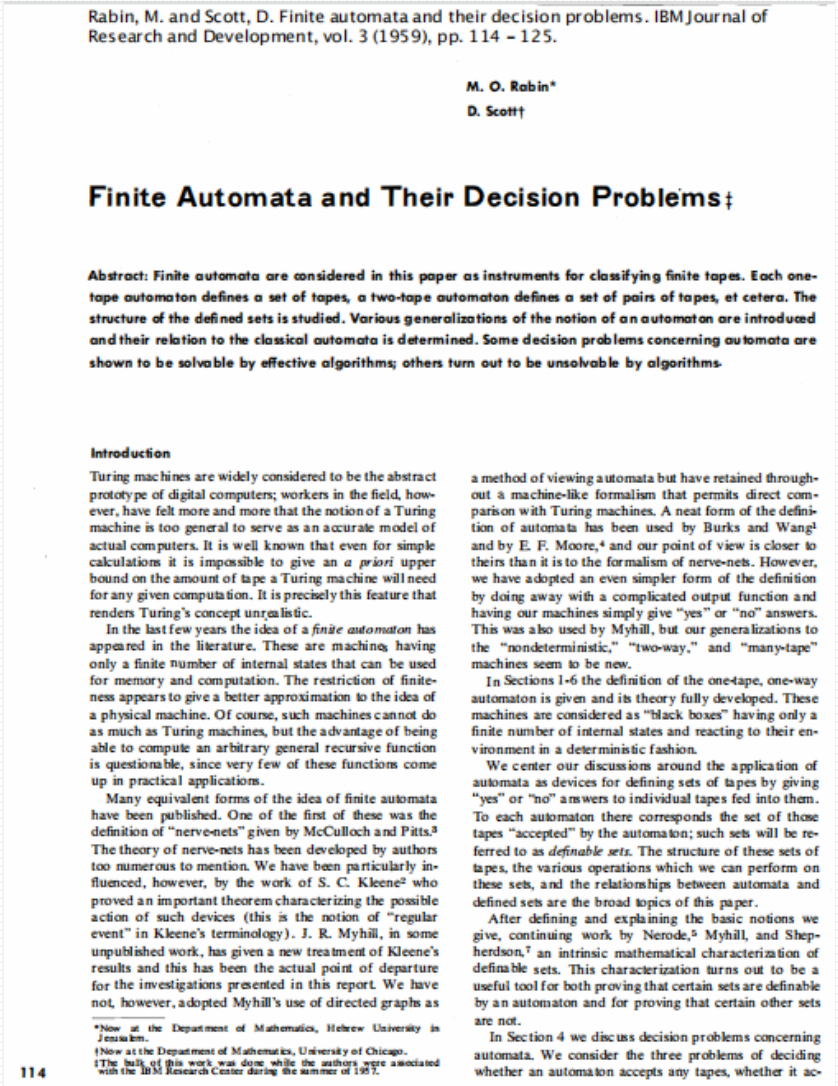
After defining and explaining the basic notions we give, continuing work by Nerode,[5] Myhill, and Shepherdson,[7] an intrinsic mathematical characterization of definable sets. This characterization turns out to be a useful tool for both proving that certain sets are definable by an automaton and for proving that certain other sets are not.

In Section 4 we discuss decision problems concerning automata. We consider the three problems of deciding whether an automaton accepts any tapes, whether it ac-

*Now at the Department of Mathematics, Hebrew University in Jerusalem.
†Now at the Department of Mathematics, University of Chicago.
‡The bulk of this work was done while the authors were associated with the IBM Research Center during the summer of 1957.

- 拉宾博士学位拿到以后，于1957年进入IBM研究中心，遇到了一起后来获得图灵奖的斯科特。
- IBM研究中心允许他们做他们感兴趣的任何工作，于是拉宾和斯科特就决定联手研究图灵机。
- 图灵在研究这种机器时的基本信条是：机器在输入相同时，其“心智状态”也相同，即对于具有给定指令集的机器而言，对于一定输入的机器总是按同一方式运行的。

NFA的诞生背景



Rabin, M. and Scott, D. Finite automata and their decision problems. IBM Journal of Research and Development, vol. 3 (1959), pp. 114 – 125.

M. O. Rabin*
D. Scott†

Finite Automata and Their Decision Problems‡

Abstract: Finite automata are considered in this paper as instruments for classifying finite tapes. Each one-tape automaton defines a set of tapes, a two-tape automaton defines a set of pairs of tapes, et cetera. The structure of the defined sets is studied. Various generalizations of the notion of an automaton are introduced and their relation to the classical automata is determined. Some decision problems concerning automata are shown to be solvable by effective algorithms; others turn out to be unsolvable by algorithms.

Introduction

Turing machines are widely considered to be the abstract prototype of digital computers; workers in the field, however, have felt more and more that the notion of a Turing machine is too general to serve as an accurate model of actual computers. It is well known that even for simple calculations it is impossible to give an *a priori* upper bound on the amount of tape a Turing machine will need for any given computation. It is precisely this feature that renders Turing's concept unrealistic.

In the last few years the idea of a *finite automaton* has appeared in the literature. These are machines having only a finite number of internal states that can be used for memory and computation. The restriction of finiteness appears to give a better approximation to the idea of a physical machine. Of course, such machines cannot do as much as Turing machines, but the advantage of being able to compute an arbitrary general recursive function is questionable, since very few of these functions come up in practical applications.

Many equivalent forms of the idea of finite automata have been published. One of the first of these was the definition of "nerve-nets" given by McCulloch and Pitts.[3] The theory of nerve-nets has been developed by authors too numerous to mention. We have been particularly influenced, however, by the work of S. C. Kleene[2] who proved an important theorem characterizing the possible action of such devices (this is the notion of "regular event" in Kleene's terminology). J. R. Myhill, in some unpublished work, has given a new treatment of Kleene's results and this has been the actual point of departure for the investigations presented in this report. We have not, however, adopted Myhill's use of directed graphs as a method of viewing automata but have retained throughout a machine-like formalism that permits direct comparison with Turing machines. A neat form of the definition of automata has been used by Burks and Wang[1] and by E. F. Moore,[4] and our point of view is closer to theirs than it is to the formalism of nerve-nets. However, we have adopted an even simpler form of the definition by doing away with a complicated output function and having our machines simply give "yes" or "no" answers. This was also used by Myhill, but our generalizations to the "nondeterministic," "two-way," and "many-tape" machines seem to be new.

In Sections 1-6 the definition of the one-tape, one-way automaton is given and its theory fully developed. These machines are considered as "black boxes" having only a finite number of internal states and reacting to their environment in a deterministic fashion.

We center our discussions around the application of automata as devices for defining sets of tapes by giving "yes" or "no" answers to individual tapes fed into them. To each automaton there corresponds the set of those tapes "accepted" by the automaton; such sets will be referred to as *definable sets*. The structure of these sets of tapes, the various operations which we can perform on these sets, and the relationships between automata and defined sets are the broad topics of this paper.

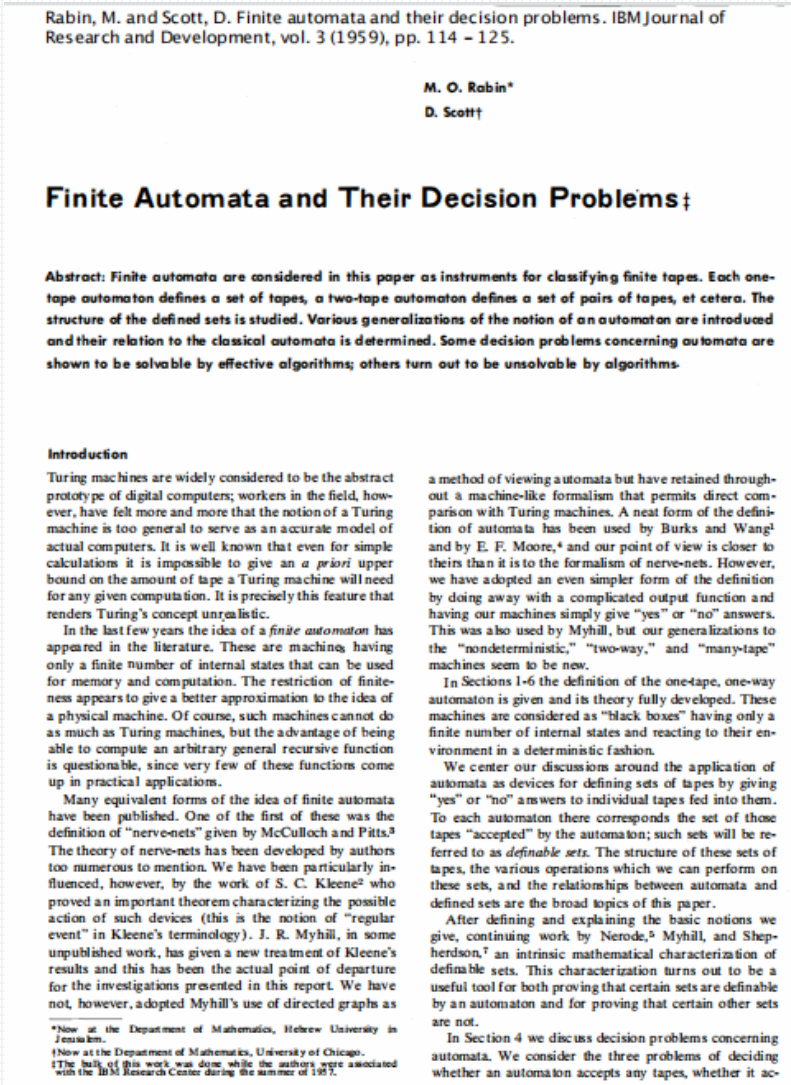
After defining and explaining the basic notions we give, continuing work by Nerode,[5] Myhill, and Shepherdson,[7] an intrinsic mathematical characterization of definable sets. This characterization turns out to be a useful tool for both proving that certain sets are definable by an automaton and for proving that certain other sets are not.

In Section 4 we discuss decision problems concerning automata. We consider the three problems of deciding whether an automaton accepts any tapes, whether it ac-

*Now at the Department of Mathematics, Hebrew University in Jerusalem.
†Now at the Department of Mathematics, University of Chicago.
‡The bulk of this work was done while the authors were associated with the IBM Research Center during the summer of 1957.

- 拉宾和斯科特认为，这种具有“确定性”行为的机器带来了局限性。因此，他们定义了一种新的、“非确定性”的有限状态自动机(nondeterministic finite state automata)，这种机器在读取到一定的输入后，有一个可以进入的新状态的"菜单"可供选择，这样对给定的输入计算便不单一了，每个选择代表一种可能的计算。
- 拉宾和斯科特将图灵的有限状态自动机从确定性一种形态扩展到非确定性的另一种形态，极大地推动了有限状态自动机理论的发展。

NFA的诞生背景



Rabin, M. and Scott, D. Finite automata and their decision problems. IBM Journal of Research and Development, vol. 3 (1959), pp. 114 – 125.

M. O. Rabin*
D. Scott†

Finite Automata and Their Decision Problems‡

Abstract: Finite automata are considered in this paper as instruments for classifying finite tapes. Each one-tape automaton defines a set of tapes, a two-tape automaton defines a set of pairs of tapes, et cetera. The structure of the defined sets is studied. Various generalizations of the notion of an automaton are introduced and their relation to the classical automata is determined. Some decision problems concerning automata are shown to be solvable by effective algorithms; others turn out to be unsolvable by algorithms.

Introduction

Turing machines are widely considered to be the abstract prototype of digital computers; workers in the field, however, have felt more and more that the notion of a Turing machine is too general to serve as an accurate model of actual computers. It is well known that even for simple calculations it is impossible to give an *a priori* upper bound on the amount of tape a Turing machine will need for any given computation. It is precisely this feature that renders Turing's concept unrealistic.

In the last few years the idea of a *finite automaton* has appeared in the literature. These are machines having only a finite number of internal states that can be used for memory and computation. The restriction of finiteness appears to give a better approximation to the idea of a physical machine. Of course, such machines cannot do as much as Turing machines, but the advantage of being able to compute an arbitrary general recursive function is questionable, since very few of these functions come up in practical applications.

Many equivalent forms of the idea of finite automata have been published. One of the first of these was the definition of "nerve-nets" given by McCulloch and Pitts.[3] The theory of nerve-nets has been developed by authors too numerous to mention. We have been particularly influenced, however, by the work of S. C. Kleene[2] who proved an important theorem characterizing the possible action of such devices (this is the notion of "regular event" in Kleene's terminology). J. R. Myhill, in some unpublished work, has given a new treatment of Kleene's results and this has been the actual point of departure for the investigations presented in this report. We have not, however, adopted Myhill's use of directed graphs as a method of viewing automata but have retained throughout a machine-like formalism that permits direct comparison with Turing machines. A neat form of the definition of automata has been used by Burks and Wang[1] and by E. F. Moore,[4] and our point of view is closer to theirs than it is to the formalism of nerve-nets. However, we have adopted an even simpler form of the definition by doing away with a complicated output function and having our machines simply give "yes" or "no" answers. This was also used by Myhill, but our generalizations to the "nondeterministic," "two-way," and "many-tape" machines seem to be new.

In Sections 1-6 the definition of the one-tape, one-way automaton is given and its theory fully developed. These machines are considered as "black boxes" having only a finite number of internal states and reacting to their environment in a deterministic fashion.

We center our discussions around the application of automata as devices for defining sets of tapes by giving "yes" or "no" answers to individual tapes fed into them. To each automaton there corresponds the set of those tapes "accepted" by the automaton; such sets will be referred to as *definable sets*. The structure of these sets of tapes, the various operations which we can perform on these sets, and the relationships between automata and defined sets are the broad topics of this paper.

After defining and explaining the basic notions we give, continuing work by Nerode,[5] Myhill, and Shepherdson,[7] an intrinsic mathematical characterization of definable sets. This characterization turns out to be a useful tool for both proving that certain sets are definable by an automaton and for proving that certain other sets are not.

In Section 4 we discuss decision problems concerning automata. We consider the three problems of deciding whether an automaton accepts any tapes, whether it ac-

*Now at the Department of Mathematics, Hebrew University in Jerusalem.
†Now at the Department of Mathematics, University of Chicago.
‡The bulk of this work was done while the authors were associated with the IBM Research Center during the summer of 1957.

1959年，拉宾和达纳·斯科特共同发表了“有限自动机与其判定性问题”（*Finite Automata and Their Decision Problems*）的论文，提出了非确定自动机的观点。他们也因此获得了1976年的图灵奖，并做“计算机复杂性”（Complexity of Computations）的演讲。图灵奖的引文是：

> “因他们的合著论文“有限自动机与其判定性问题”。论文中引入了非确定自动机的概念，被证明是（计算理论科学研究中的）一个非常重要的概念。拉宾和斯科特的这篇经典论文成为了这个领域后续研究的源泉。[1]”

非确定有穷自动机

- 例:由0和1构成的串中，接受全部以01结尾的串，如何设计有穷自动机?

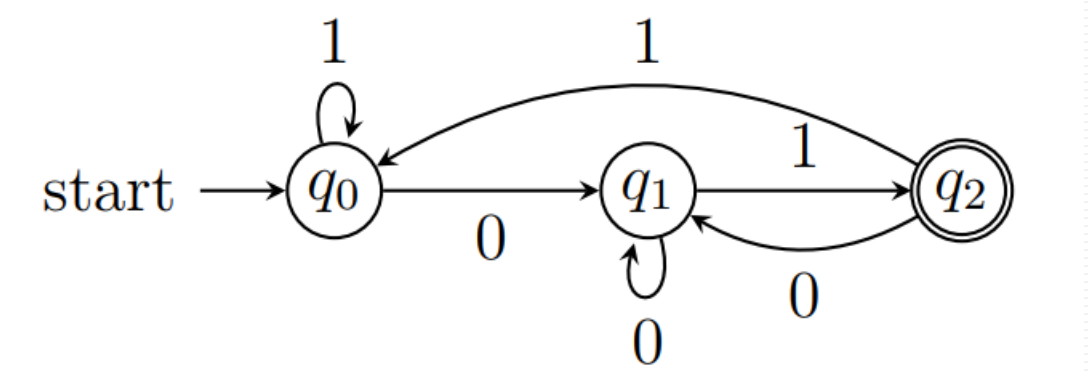
DFA

- **例:由0和1构成的串中，接受全部以01结尾的串，如何设计有穷自动机?**
  - 思路：要识别以01结尾的符合串至少要有3个状态:
    $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \rightarrow$
    - 开始状态为q_0，识别了一个0之后转移到状态q_1，再识别一个1之后转移到状态q_2
    - q_2为接受状态，可以接受的符号串在识别完01之后就应该结束
  - 如果q_2之后还有字符输入，说明符号串还未结束，需要添加状态转移过程，从q_2进行跳转。
  - 处于q_2时，如果继续输入的是0，则转移到q_1；如果输入的是1，则转移到q_0

DFA

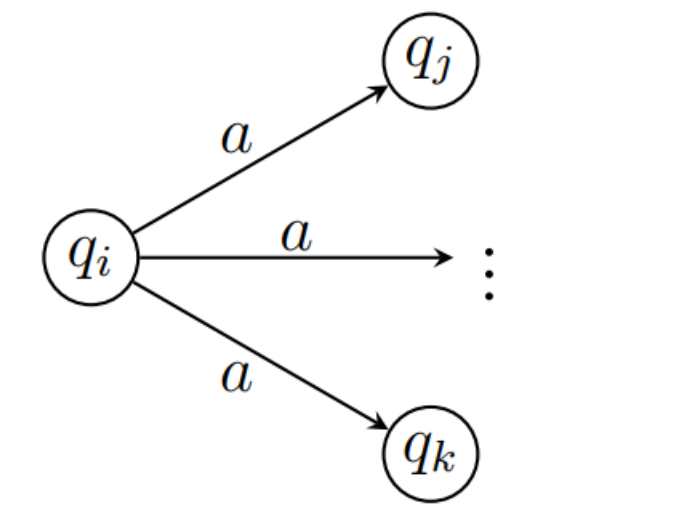
- 例:由0和1构成的串中，接受全部以01结尾的串，如何设计有穷自动机?

DFA



能否进行简化?

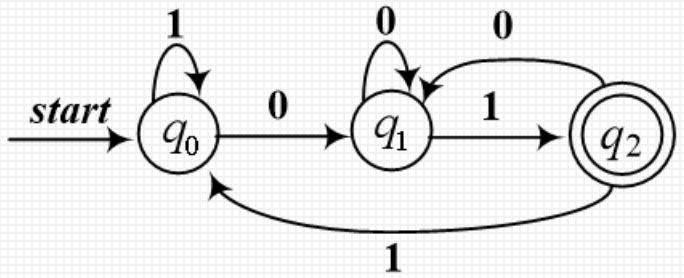
状态的非确定转移



- **同一状态在相同的输入下，可以有多个转移状态**
- **自动机可以处于多个当前状态**
  - 因此给定一个输入和当前状态，状态转移函数的输出是一个状态集。

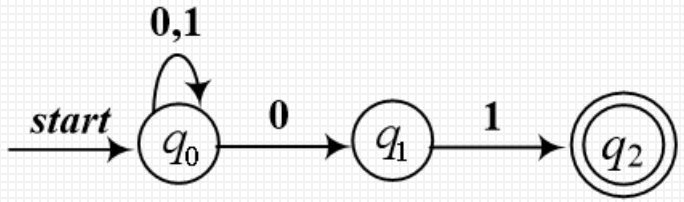
- **续例:由0和1构成的串中，接受全部以01结尾的串。**

确定的



	0	1
→q_0	q_1	q_0
q_1	q_1	q_2
*q_2	q_1	q_0

非确定

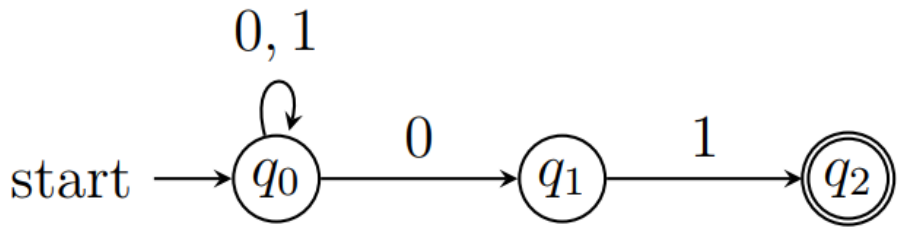


	0	1
→q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	Φ	$\{q_2\}$
*q_2	Φ	Φ

非确定有穷自动机运转模式

- **续例:由0和1构成的串中，接受全部以01结尾的串。**

非确定

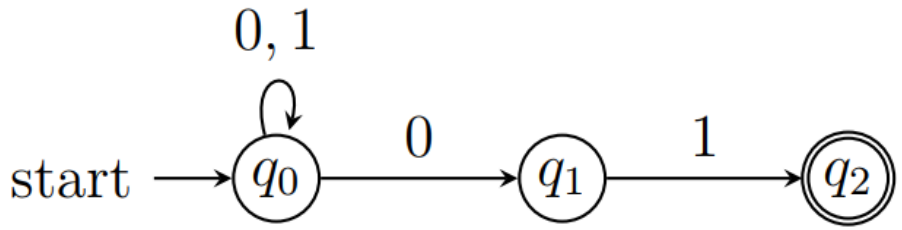


- **运转模式：**
  - 开始处于q_0,如果读入1，则状态保持在q_0；

非确定有穷自动机运转模式

- **续例:由0和1构成的串中，接受全部以01结尾的串。**

非确定

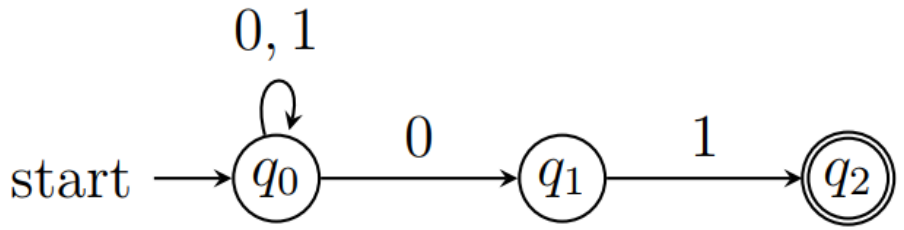


- **运转模式:**
  - 开始处于q_0,如果读入1，则状态保持在q_0;
  - 读入0的话，处于两个状态，q_0和q_1;

非确定有穷自动机运转模式

- **续例:由0和1构成的串中，接受全部以01结尾的串。**

非确定

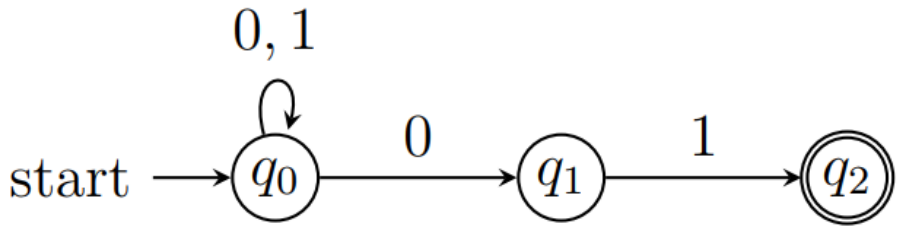


- **运转模式:**
  - 开始处于q_0,如果读入1，则状态保持在q_0;
  - 读入0的话，处于两个状态，q_0和q_1;
  - 并且在q_1的时候，只有读入1，才可以跳转到q_2;
  - 即，只有以01结尾，才可能跳转到状态q_2。

非确定有穷自动机运转模式

- **续例:由0和1构成的串中，接受全部以01结尾的串。**

非确定



- **以读入101为例：**
  - 开始处于q_0,读入1，则状态还是q_0;
  - 接下来读入0，跳转到两个状态，q_0和q_1;
  - 接下来读入1，则$q_0 \to q_0, q_1 \to q_2$，即处于q_0和q_2;
  - 由于最后的状态中包含有接受状态q_2，该串被接受。

非确定有穷自动机的形式化定义

- **定义：非确定有穷自动机(NFA, Non-deterministic Finite Automaton)A为五元组**$(Q, \Sigma, \delta, q_0, F)$

(1) Q:有穷状态集

(2) Σ: 有穷输入符号集或字母表

(3) $\delta: Q \times \Sigma \longrightarrow 2^Q$:状态转移函数

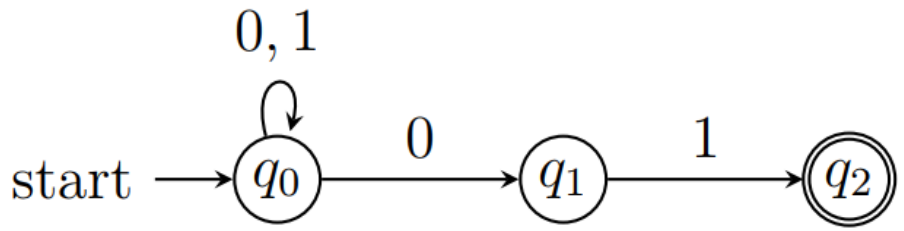
(4) $q_0 \in Q$: 初始状态

(5) $F \subseteq Q$: 终结状态集或接受状态集。

注:2^Q是幂集(幂集是一个集合中所有的子集构成的集合)，即2^Q表示Q的所有子集构成的集合，每个子集视作一个元素。

NFA的形式定义示例

- **续例：接受全部以 01 结尾的串的NFA。**



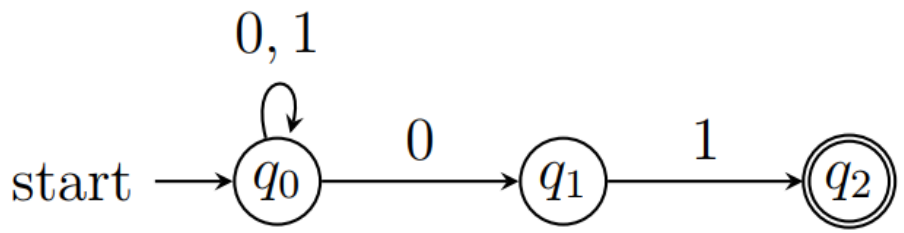
五元组为 $A=(\{q_0,q_1,q_2\},\{0,1\},\delta,q_0,\{q_2\})$ **，转移函数** δ**：**

$$\delta(q_0,0)=\{q_0,q_1\} \quad \delta(q_1,0)=\varnothing \quad \delta(q_2,0)=\varnothing$$

$$\delta(q_0,1)=\{q_0\} \quad \delta(q_1,1)=\{q_2\} \quad \delta(q_2,1)=\varnothing$$

NFA的形式定义示例

- 续例：接受全部以 01 结尾的串的NFA。



- 状态转移表：

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

扩展状态转移函数

- **定义. 与DFA类似，扩展δ到字符串，定义扩展状态转移函数 $\hat{\delta}: Q \times \Sigma^* \to 2^Q$为**

$$\hat{\delta}(q,w) = \begin{cases} \{q\}, & w = \epsilon \\ \bigcup_{p \in \hat{\delta}(q,x)} \delta(p,a), & w = xa \end{cases}$$

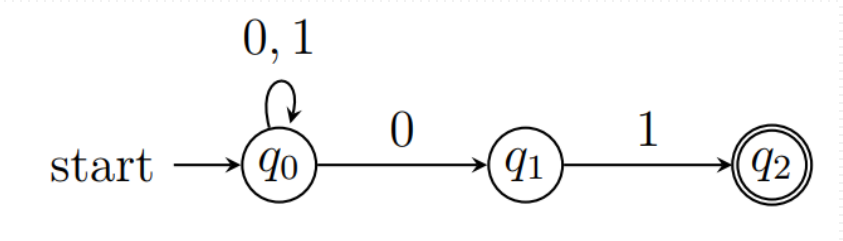
DFA的扩展状态转移函数：

$$\hat{\delta}(q,w) = \begin{cases} q, & w = \epsilon \\ \delta\left(\hat{\delta}(q,x), a\right), & w = xa \end{cases}$$

其中，a是一个字符，w和x是字符串。

当w不为空串的时候，先输入前面的字符串x，可以到达多个状态 $\hat{\delta}(q,x)$ 。对每个状态应用状态转移函数，然后把状态求并集即可。

- **例. 接受01结尾的串的NFA, $\hat{\delta}$处理00101时每步的状态转移**



$$\hat{\delta}(q,w)=\begin{cases}\{q\}, & w=\epsilon\\ \cup_{p\in\hat{\delta}(q,x)}\delta(p,a), & w=xa\end{cases}$$

1 $\hat{\delta}(q_0,\varepsilon)=\{q_0\}$

2 $\hat{\delta}(q_0,0)=\delta(q_0,0)=\{q_0,q_1\}$

3 $\hat{\delta}(q_0,00)=\delta(q_0,0)\cup\delta(q_1,0)=\{q_0,q_1\}\cup\varnothing=\{q_0,q_1\}$

4 $\hat{\delta}(q_0,001)=\delta(q_0,1)\cup\delta(q_1,1)=\{q_0\}\cup\{q_2\}=\{q_0,q_2\}$

5 $\hat{\delta}(q_0,0010)=\delta(q_0,0)\cup\delta(q_2,0)=\{q_0,q_1\}\cup\varnothing=\{q_0,q_1\}$

6 $\hat{\delta}(q_0,00101)=\delta(q_0,1)\cup\delta(q_1,1)=\{q_0\}\cup\{q_2\}=\{q_0,q_2\}$

最后含有接受状态，00101可以被接受。

NFA的语言

- 回顾：若$\boldsymbol{D = (Q, \Sigma, \delta, q_0, F)}$是一个DFA，则$\boldsymbol{D}$接受的语言为

$$\mathbf{L}(D) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\}$$

- 定义：若$\boldsymbol{N = (Q, \Sigma, \delta, q_0, F)}$是一个NFA，则$\boldsymbol{N}$接受的语言为

$$\mathbf{L}(N) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

DFA和NFA对比

DFA

- 状态的转移是确定的
  - 一次状态转移只会转移到一个确定的状态
- 对于每一个状态，需要明确定义对应于各个可能的输入符号的状态转移
- 有时设计较复杂
- 最后所处的状态在接受状态集F中时，才接受符号串。

NFA

- 状态的转移是非确定的
  - 一次状态转移可能转移到多个状态，产生多条转移路径
- 不需要明确地定义所有的状态转移（如果未定义的话那么会进入一个dead state）
- 通常比DFA更为容易设计
- 最后所处的状态集中有某个状态在接受状态集F中时，才接受符号串。

但是，DFA和NFA表示语言的能力是等价的。

DFA与NFA的等价性

- 定理:如果语言L被NFA接受，当且仅当L被DFA接受。
- DFA是NFA的特例，所以NFA必然能接收DFA能接收的语言．因此证明等价性只要能够证明一个NFA所能接收的语言必能被另一个DFA所接收。
- 具体的，给定一个NFA的五元组，构造一个DFA，然后证明它们是等价的。
  - 通过子集构造法（subset construction）进行。

从NFA到DFA：子集构造法

- **目标：NFA** $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$**构造** $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$，且$\mathbf{L}(D) = \mathbf{L}(N)$。
- **构造**:
  - 状态集： $Q_D = 2^{Q_N}$,即 $\boldsymbol{Q_D}$**中的每个状态元素是**$\boldsymbol{Q_N}$**的一个子集**
  - 接受状态集：$F_D = \{S | S \subset Q_N, S \cap F_N \neq \emptyset\}$ (set of subsets S of Q_N)
    - F_D的每一个状态元素亦是$\boldsymbol{Q_N}$**的一个子集，且这些子集与**F_N的交集不为空。
  - 状态转移函数: $\forall S \subset Q_N, \forall a \in \Sigma, \delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$。

例如，假设DFA的某个状态为$\{q_1, q_2, q_3\}$，输入符号a,那么$\delta_D(\{q_1, q_2, q_3\}, \mathrm{a})$是$\delta_N(q_i, a)$的并集$(i = 1,2,3)$。

（注意，这里$\{q_1, q_2, q_3\}$表示DFA的一个状态，是状态名，而不是直接看作一个集合。）

归纳法证明

- 用归纳法证明$\mathbf{L}(\boldsymbol{D}) = \mathbf{L}(\boldsymbol{N})$，对串$\boldsymbol{w}$的长度进行归纳，往证$\hat{\boldsymbol{\delta}}_{\boldsymbol{D}}(\{\boldsymbol{q_0}\}, \boldsymbol{w}) = \hat{\boldsymbol{\delta}}_{\boldsymbol{N}}(\boldsymbol{q_0}, \boldsymbol{w})$，即输入相同的字符串能够从开始状态转移相同的状态。
- 归纳基础：当$|\boldsymbol{w}| = \mathbf{0}$, 即$\boldsymbol{w} = \boldsymbol{\varepsilon}$,

$$\hat{\delta}_D(\{q_0\}, \varepsilon) = \{q_0\} = \hat{\delta}_N(q_0, \varepsilon)$$

$$\widehat{\delta}_D(\{q_0\}, w) = \widehat{\delta}_N(q_0, w)$$

- 归纳假设：假设$|w| = n$时，命题成立。
- 归纳递推：当$|w| = n + 1$时，则$w = xa$,

状态转移函数定义:
$\forall S \subset Q_N, \forall a \in \Sigma,$
$\delta_D(S, a) = \cup_{p \in S} \delta_N(p, a)$

$$\begin{aligned}
\hat{\delta}_N(q_0, xa) &= \cup_{p \in \widehat{\delta}_N(q_0, x)} \delta_N(p, a) && \text{NFA的}\delta\text{定义} \\
&= \cup_{p \in \widehat{\delta}_D(\{q_0\}, x)} \delta_N(p, a) && \text{归纳假设} \\
&= \delta_D\left(\hat{\delta}_D(\{q_0\}, x), a\right) && D\text{的状态转移函数定义} \\
&= \hat{\delta}_D(\{q_0\}, xa) && \text{DFA的}\delta\text{定义}
\end{aligned}$$

即此处 $S = \hat{\delta}_D(\{q_0\}, x)$

因此上式成立。

DFA与NFA的等价性证明

- **因为** $\hat{\delta}_D(\{q_0\}, w) = \hat{\delta}_N(q_0, w)$,
- **所以**

$$w \in \boldsymbol{L}(N) \Leftrightarrow \hat{\delta}_N(q_0, w) \bigcap F_N \neq \varnothing \qquad \text{NFA的语言}$$

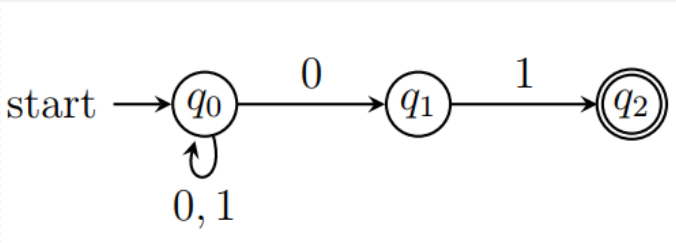
$$\Leftrightarrow \hat{\delta}_D(\{q_0\}, w) \bigcap F_N \neq \varnothing \qquad \text{刚证明的}$$

$$\Leftrightarrow \hat{\delta}_D(\{q_0\}, w) \in F_D \qquad \text{D的构造}$$

$$\Leftrightarrow w \in \boldsymbol{L}(D) \qquad \text{DFA的语言}$$

所以 $\mathbf{L}(D) = \mathbf{L}(N)$。

- **例:构造接受全部以 01 结尾的串的NFA。**



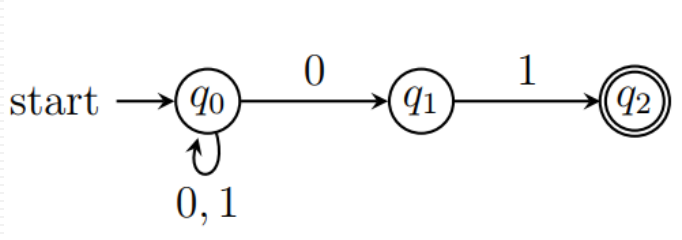
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

1. 列出DFA的所有可能状态集(为NFA状态集的所有子集)。

	0	1
$\emptyset$		
$\rightarrow \{q_0\}$		
$\{q_1\}$		
$\{q_2\}$		

	0	1
$\{q_0, q_1\}$		
$\{q_0, q_2\}$		
$\{q_1, q_2\}$		
$\{q_0, q_1, q_2\}$		

- **例:构造接受全部以 01 结尾的串的NFA。**



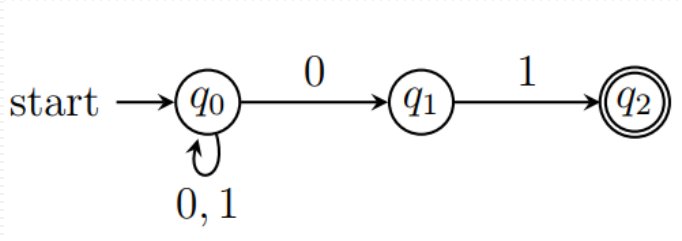
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

2. 转移状态为NFA中相应状态的并集。

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$

	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

- **例:构造接受全部以 01 结尾的串的NFA。**



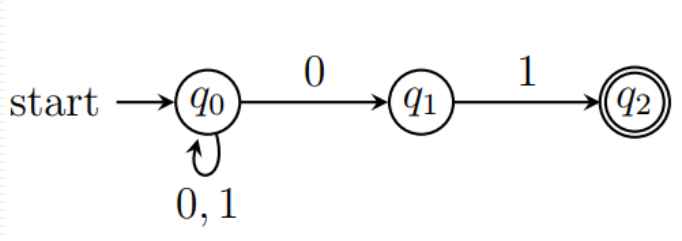
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

3. 删除含有∅的状态，删除不能从开始状态达到的状态。

	0	1
~~$\emptyset$~~	~~$\emptyset$~~	~~$\emptyset$~~
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
~~$\{q_1\}$~~	~~$\emptyset$~~	~~$\{q_2\}$~~
~~$\{q_2\}$~~	~~$\emptyset$~~	~~$\emptyset$~~

	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
~~$\{q_1, q_2\}$~~	~~$\emptyset$~~	~~$\{q_2\}$~~
~~$\{q_0, q_1, q_2\}$~~	~~$\{q_0, q_1\}$~~	~~$\{q_0, q_2\}$~~

- **例:构造接受全部以 01 结尾的串的NFA。**



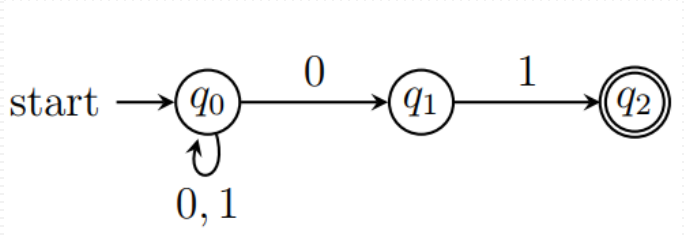
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

4. 含有NFA接受状态的状态集就是DFA的接受状态。

	0	1
~~$\emptyset$~~	~~$\emptyset$~~	~~$\emptyset$~~
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
~~$\{q_1\}$~~	~~$\emptyset$~~	~~$\{q_2\}$~~
~~$\{q_2\}$~~	~~$\emptyset$~~	~~$\emptyset$~~

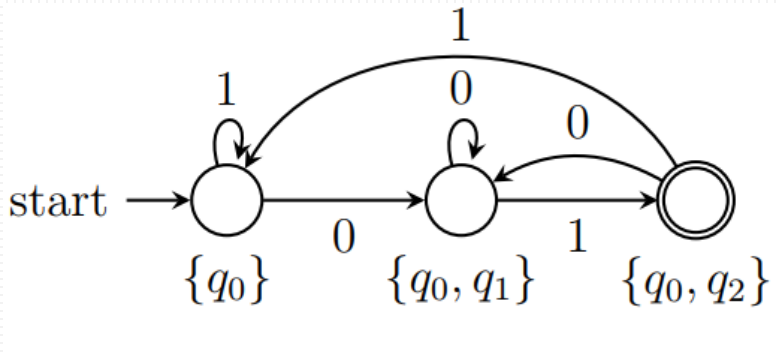
	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
~~$\{q_1, q_2\}$~~	~~$\emptyset$~~	~~$\{q_2\}$~~
~~$\{q_0, q_1, q_2\}$~~	~~$\{q_0, q_1\}$~~	~~$\{q_0, q_2\}$~~

- **例:构造接受全部以 01 结尾的串的NFA。**



	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

4. 含有NFA接受状态的状态集就是DFA的接受状态。

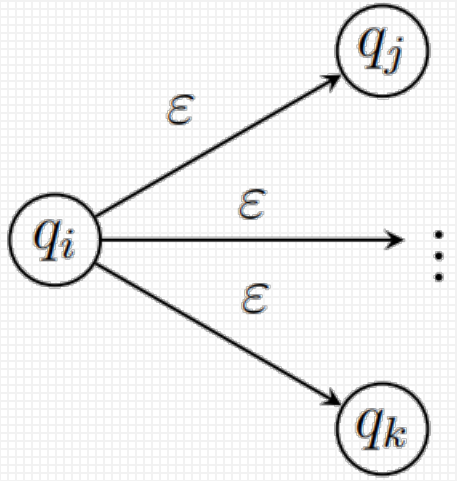


主要内容

- 确定的有穷自动机
- 非确定有穷自动机
- **带空转移的非确定有穷自动机**

带有空转移的NFA(ε-NFA)

- **ε-NFA：对NFA进行扩展，在不输入任何字符的情况下(空串 ε)，可以自发地发生状态转移。**
  - 可以简化NFA的构造
  - ε-NFA是明确地定义了至少一个空转移的NFA
  - 在状态表里面多了对应于ε的一列



ε-NFA的形式化定义

- **定义：带空转移非确定有穷自动机(ε-NFA) A 为五元组$A = (Q, \Sigma, \delta, q_0, F)$。**

(1) Q:有穷状态集；

(2) Σ: 有穷输入符号集或字母表；

(3) $\delta: Q \times (\Sigma \cup \varepsilon) \longrightarrow 2^Q$:状态转移函数；

(4) $q_0 \in Q$: 初始状态

(5) $F \subseteq Q$: 终结状态集或接受状态集。

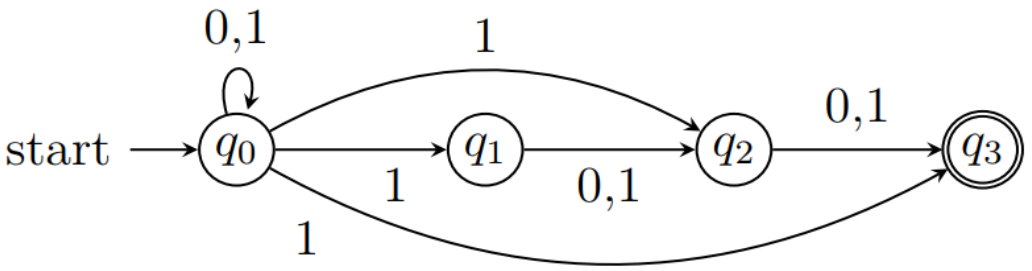
注意：

此后除非特别申明，不再明确区分ε-NFA和NFA，而是都称为NFA。

ε-NFA示例

- **例**：$L = \{w \in \{0,1\}^* \mid w$ 倒数 3个字符至少有一个是 $1\}$**的NFA。**

  不带空转移的NFA：

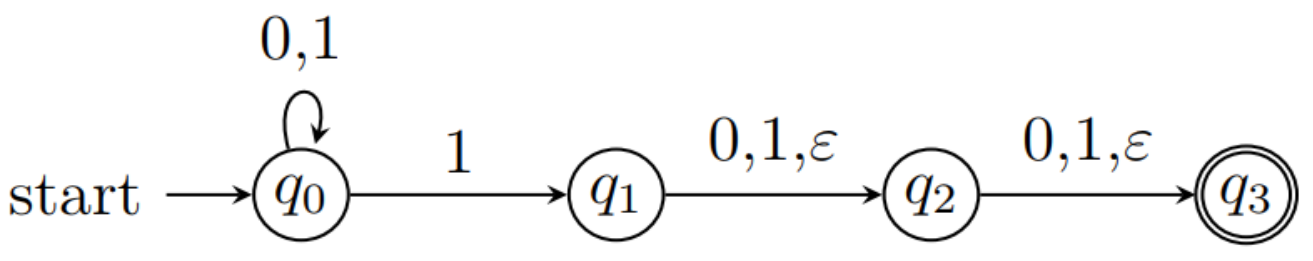


- 在q_0循环输入所有的字符，
  - 当倒数第一个是1时，最终会跳转到q_3。
  - 当倒数第二是1时，就会跳转到q_2（后续不管输入0或1，都会到q_3）。
  - 当倒数第三个是1时，就会跳转到q_1（后续不管输入0或1，都会到q_3）。

ε-NFA示例

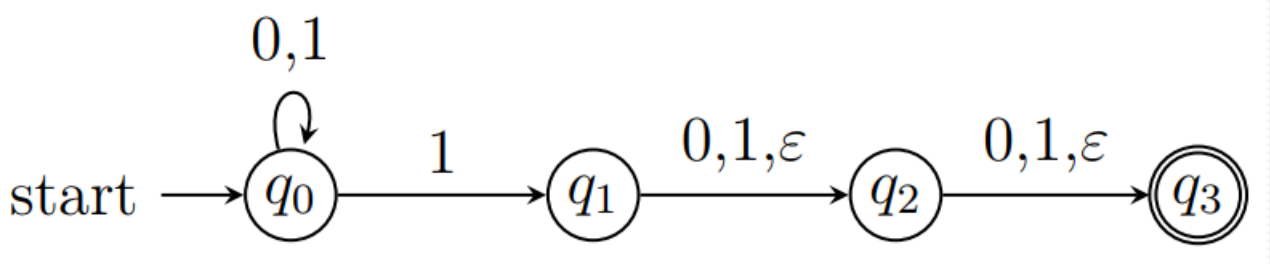
- **例**：$L = \{w \in \{0,1\}^* \mid w$ 倒数 3个字符至少有一个是 $1\}$**的NFA。**

引入空转移的 **ε-NFA**：

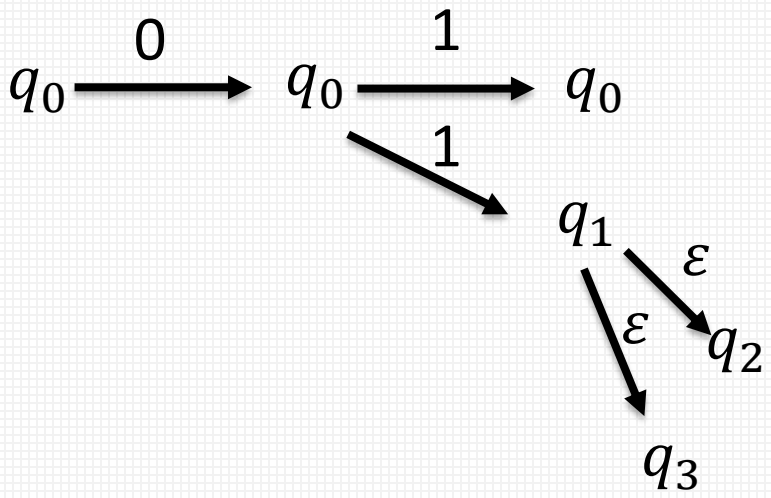


- 在q_0循环输入所有的字符，
  - 只要倒数3个字符里面至少有一个1，总能先跳到q_1，这样即使字符已经消耗完，也可以通过空转移跳到q_3，进入接受状态。

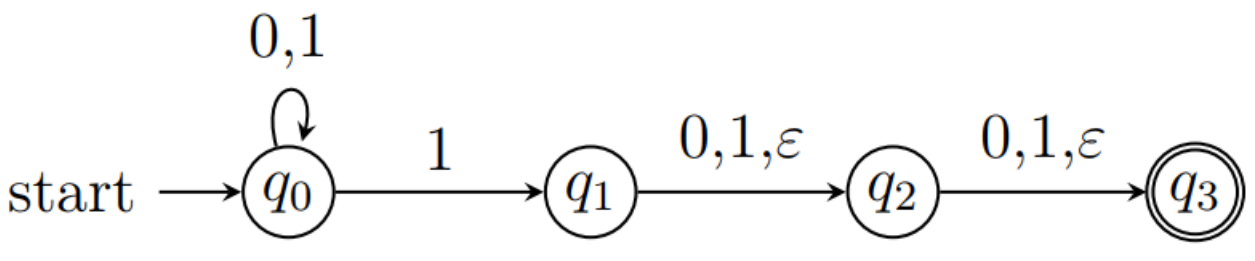
ε-NFA示例



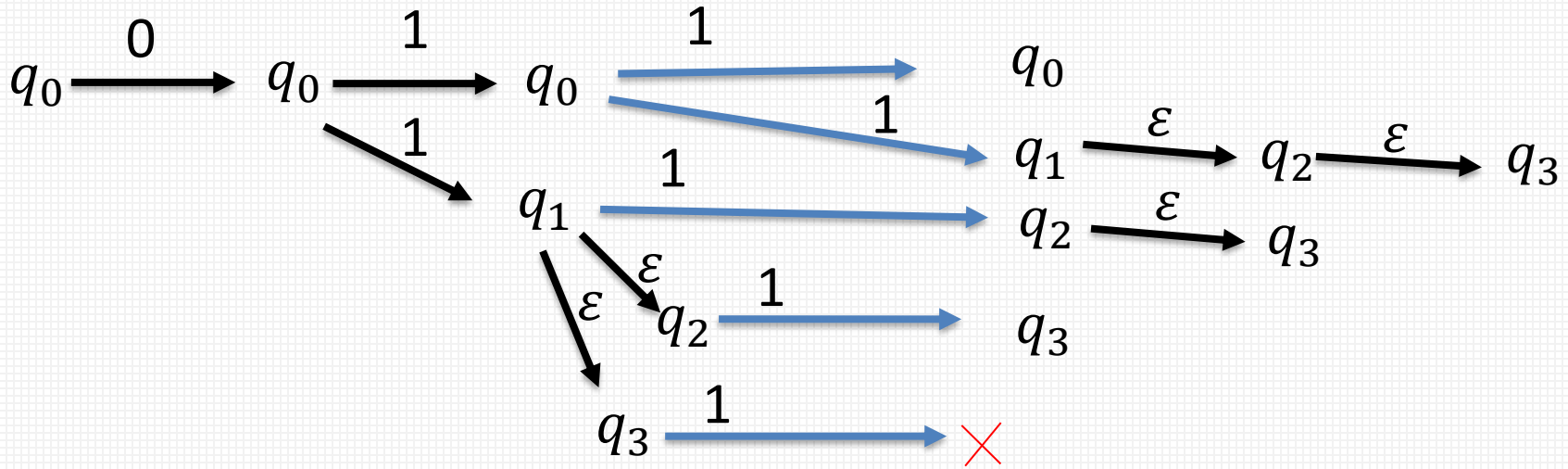
- 以输入011为例，



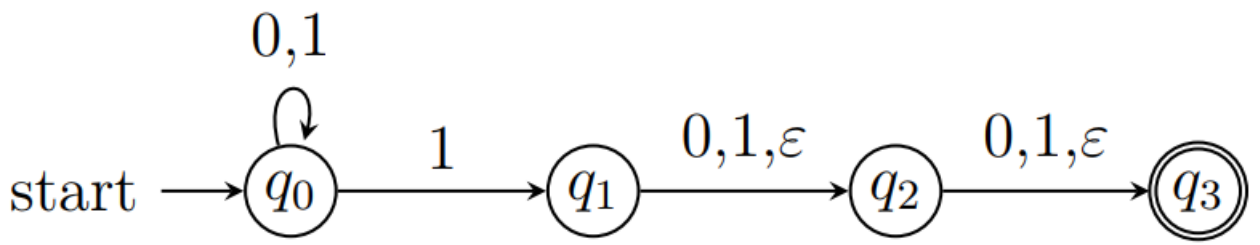
ε-NFA示例



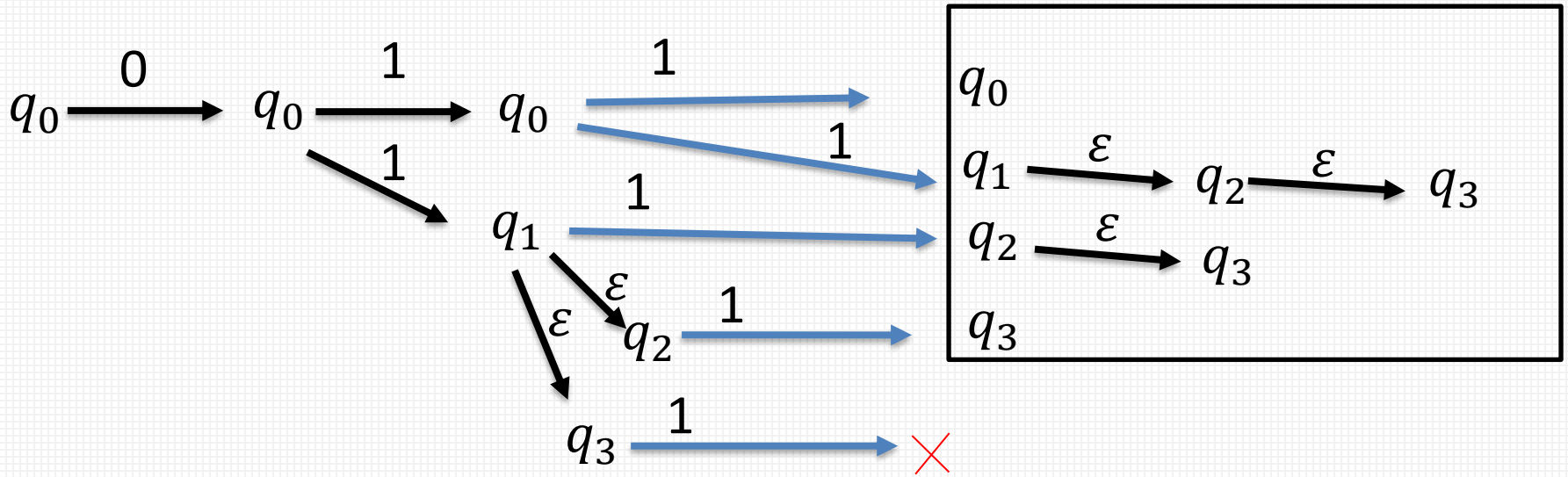
- 以输入011为例，



ε-NFA示例



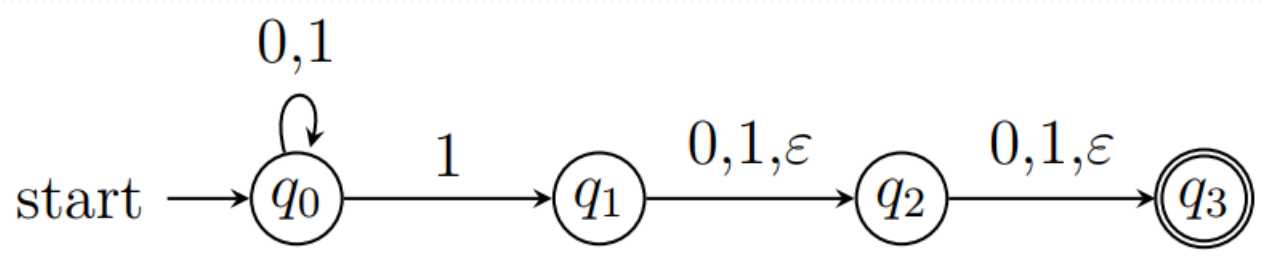
- 以输入011为例，



最终所处的状态为q_0, q_1, q_2, q_3(011可被接受)。

状态转移表

- **例续：$L = \{w \in \{0,1\}^* \mid w$ 倒数 3个字符至少有一个是 1$\}$的ε-NFA的状态转移表。**



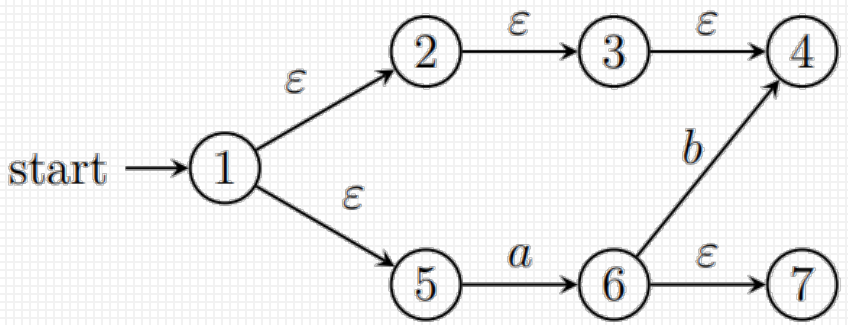
	0	1	ε
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$
$*q_3$	$\emptyset$	$\emptyset$	$\emptyset$

状态的ε-闭包

- **定义：状态q的ε-闭包（ε-closure），记为$\mathbf{ECLOSE}(q)$，表示从q经过$\varepsilon\varepsilon\cdots\varepsilon$序列可到达的全部状态的集合。（即经过0个或>0个空转移到达的状态集合）**
- **递归定义：**

(1)$q \in \mathrm{ECLOSE}(q)$;（包含当前状态本身）

(2)$\forall p \in \mathrm{ECLOSE}(q)$, 若$r \in \delta(p, \varepsilon)$, 则$r \in \mathrm{ECLOSE}(q)$

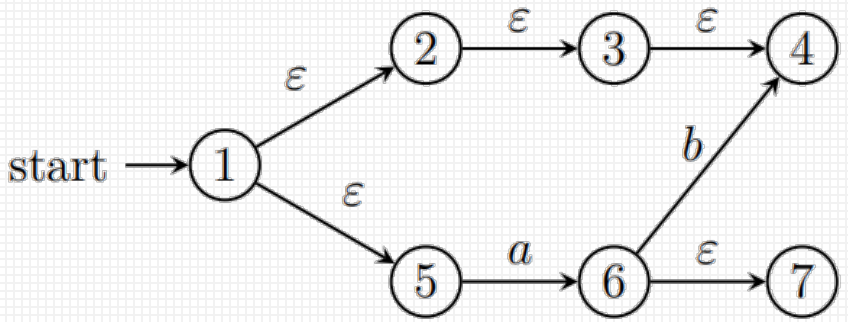


状态的ε-闭包

- **定义：状态q的ε-闭包（ε-closure），记为$\mathbf{ECLOSE}(q)$，表示从q经过$\varepsilon\varepsilon\cdots\varepsilon$序列可到达的全部状态的集合。（即经过0个或>0个空转移到达的状态集合）**
- **递归定义：**

(1)$q \in \mathrm{ECLOSE}(q)$;（包含当前状态本身）

(2)$\forall p \in \mathrm{ECLOSE}(q)$, 若$r \in \delta(p, \varepsilon)$, 则$r \in \mathrm{ECLOSE}(q)$



$\mathrm{ECLOSE}(1) = \{1,2,5,3,4\}$

$\mathrm{ECLOSE}(2) = \{2,3,4\}$

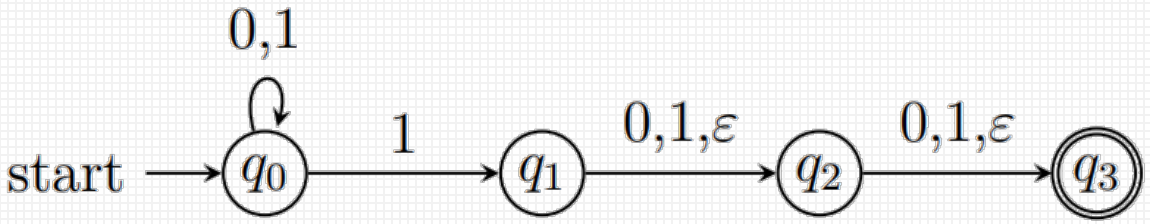
状态集的ε-闭包

- **定义：状态集S的ε-闭包为**

$$\mathrm{ECLOSE}(S) = \cup_{q \in S} \mathrm{ECLOSE}(q)$$

即，一个状态集S的ε-闭包为S里面每个状态的ε-闭包的并集。

- **续例：$L = \{w \in \{0,1\}^* \mid w$ 倒数3个字符至少有一个是1$\}$的NFA。**



- **状态转移表及每个状态的ε-闭包：**

	0	1	ε	ECLOSE(␣)
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_1, q_2, q_3\}$
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_2, q_3\}$
$*q_3$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$

扩展转移函数

- **定义：扩展δ到字符串，定义扩展状态转移函数$\hat{\delta}: \mathbf{Q} \times \boldsymbol{\Sigma}^* \rightarrow 2^{Q}$为：**

$$\hat{\delta}(q,w) = \begin{cases} \text{ECLOSE}(q), & w = \epsilon \\ \text{ECLOSE}\left(\cup_{p \in \hat{\delta}(q,x)} \delta(p,a)\right) & w = xa \end{cases}$$

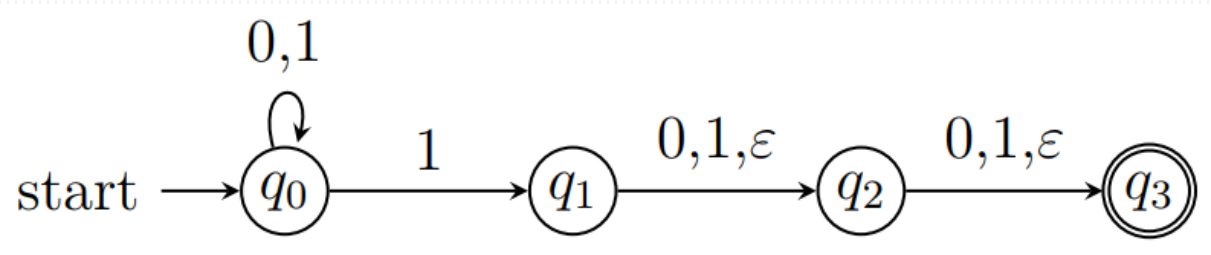
常规NFA的扩展状态转移函数：

$$\hat{\delta}(q,w) = \begin{cases} \{q\}, & w = \epsilon \\ \cup_{p \in \hat{\delta}(q,x)} \delta(p,a), & w = xa \end{cases}$$

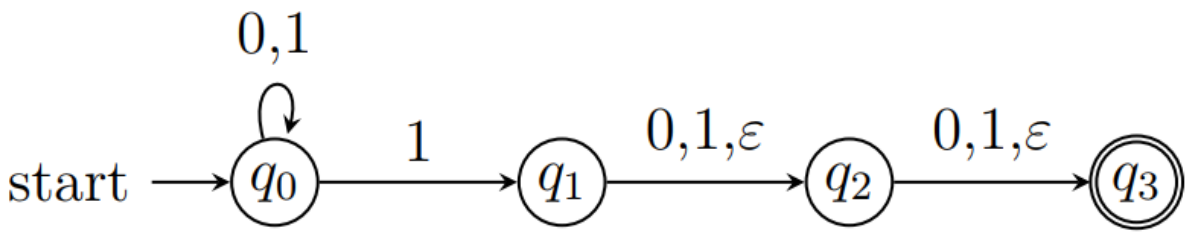
其中，a是一个字符，w和x是字符串。

扩展转移函数

- **续例**：$L = \{w \in \{0,1\}^* \mid w$ 倒数3个字符至少有一个是1$\}$**的**ε**-NFA如下，求**$\hat{\delta}(q_0, 10) =?$



扩展转移函数



$$\hat{\delta}(q,w)=\begin{cases}\text{ECLOSE}(q), & w=\epsilon\\ \text{ECLOSE}\left(\cup_{p\in\hat{\delta}(q,x)}\delta(p,a)\right) & w=xa\end{cases}$$

$$\hat{\delta}(q_0,\varepsilon)=\text{ECLOSE}(q_0)=\{q_0\}$$

$$\hat{\delta}(q_0,1)=\text{ECLOSE}\left(\cup_{p\in\hat{\delta}(q_0,\varepsilon)}\delta(p,1)\right)$$

$$=\text{ECLOSE}(\delta(q_0,1))=\text{ECLOSE}(\{q_0,q_1\})=\{q_0,q_1,q_2,q_3\}$$

$$\hat{\delta}(q_0,10)=\text{ECLOSE}\left(\cup_{p\in\hat{\delta}(q_0,1)}\delta(p,0)\right)$$

$$=\text{ECLOSE}(\delta(q_0,0)\cup\delta(q_1,0)\cup\delta(q_2,0)\cup\delta(q_3,0))$$

$$=\text{ECLOSE}(\{q_0,q_2,q_3\})=\{q_0,q_2,q_3\}$$

ε-NFA的语言

- 回顾：DFA $D=(Q,\Sigma,\delta,q_0,F)$和NFA $N=(Q,\Sigma,\delta,q_0,F)$的语言分别为

$$\mathbf{L}(D)=\{w\in\Sigma^*|\hat{\delta}(q_0,w)\in F\}$$
$$\mathbf{L}(N)=\{w\in\Sigma^*|\hat{\delta}(q_0,w)\cap F\neq\emptyset\}$$

- 定义：若$E=(Q,\Sigma,\delta,q_0,F)$是一个ε-NFA，则$E$接受的语言为

$$\mathbf{L}(E)=\{w\in\Sigma^*|\hat{\delta}(q_0,w)\cap F\neq\emptyset\}$$

ε-NFA与DFA等价性

- 定理:如果语言L被ε-NFA接受,当且仅当L被DFA接受。

构造与ε-NFA等价的DFA

- **子集构造法（消除空转移）：如果ε-NFA $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$，构造DFA**

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

(1) $Q_D = 2^{Q_E}$

(2) $q_D = \mathrm{ECLOSE}(q_E)$

(3) $F_D = \{S | S \subseteq Q_E, S \cap F_E \neq \emptyset\}$

(4) $\forall S \subseteq Q_E, \forall a \in \Sigma,$

$$\delta_D(S, a) = \mathrm{ECLOSE}(\cup_{p \in S} \delta_E(p, a))$$

那么$\mathbf{L}(D) = \mathbf{L}(E)$。

- **定理：如果语言L被ε-NFA接受，当且仅当L被DFA接受。**
- **证明：对串w的长度进行归纳，往证$\hat{\delta}_E(q_E, w) = \hat{\delta}_D(q_D, w)$。**

(1) 当$w = \varepsilon$时，$\hat{\delta}_E(q_E, \varepsilon) = \mathrm{ECLOSE}(q_E), q_D = \hat{\delta}_D(q_D, \varepsilon)$，而$q_D = \mathrm{ECLOSE}(q_E)$，因此$\hat{\delta}_E(q_E, \varepsilon) = \hat{\delta}_D(q_D, \varepsilon)$

(2) 假设当$w = x$时，$\hat{\delta}_E(q_E, x) = \hat{\delta}_D(q_D, x)$

(3) 则当$w = xa$时，

$$\hat{\delta}_E(q_E, xa) = \mathrm{ECLOSE}\left(\cup_{p \in \hat{\delta}_E(q_E, x)} \delta_E(p, a)\right)$$

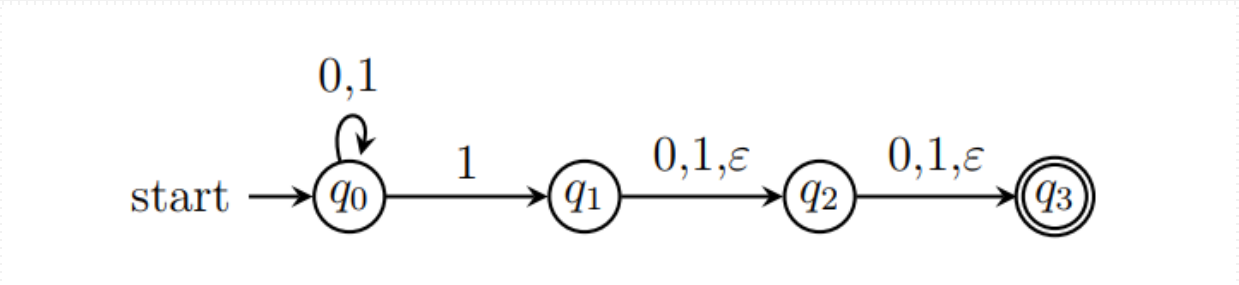
$$= \mathrm{ECLOSE}\left(\cup_{p \in \hat{\delta}_D(q_D, x)} \delta_E(p, a)\right)$$ 由归纳假设

$$= \delta_D(\hat{\delta}_D(q_D, x), a)$$ 所构造的DFA的转移函数定义

$$= \hat{\delta}_D(q_D, xa)$$

构造与ε-NFA等价的DFA

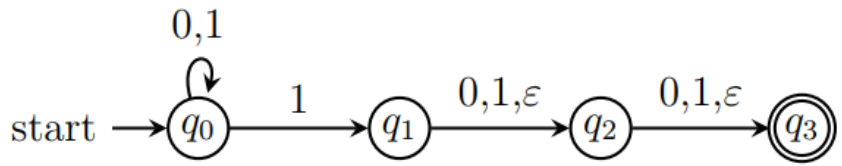
- 例:将下图L的ε-NFA，转为等价的DFA。



	0	1	ε	ECLOSE()
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_1, q_2, q_3\}$
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_2, q_3\}$
$*\, q_3$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$

构造与ε-NFA等价的DFA

- 子集构造法



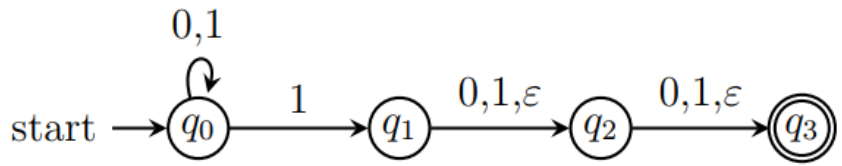
  - DFA

	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
q_1	$\{q_2, q_3\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
$*\, q_3$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_1, q_2\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$

	0	1
$*\{q_1, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_2, q_3\}$	$\{q_3\}$	$\{q_3\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_1, q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\emptyset$	$\emptyset$	$\emptyset$

构造与ε-NFA等价的DFA

- 子集构造法
  - DFA

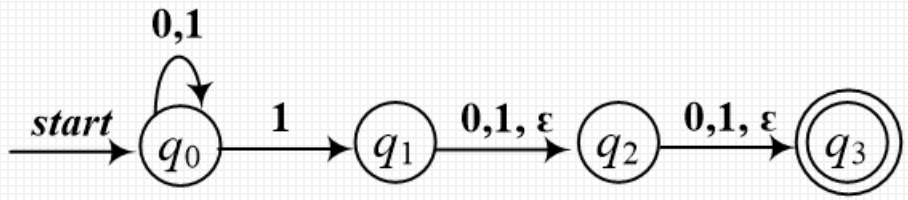


	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
q_1	$\{q_2, q_3\}$	$\{q_2\}$
q_2	$\{q_3\}$	$\{q_3\}$
$*\ q_3$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_1, q_2\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$

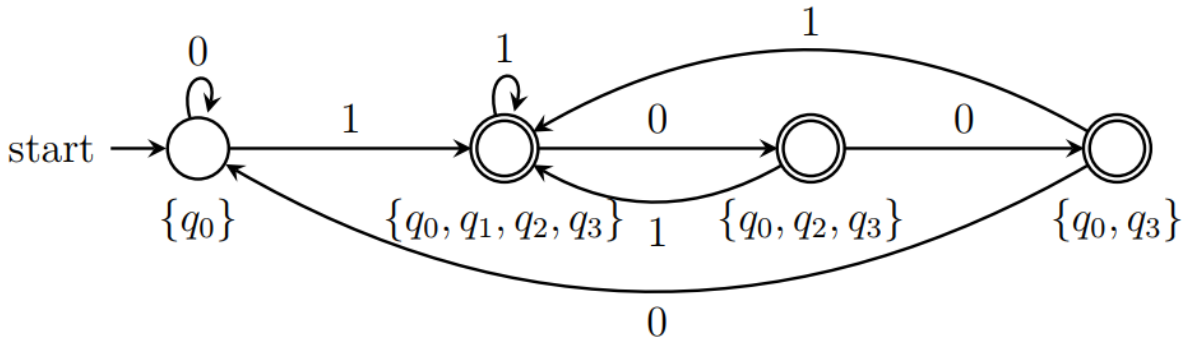
	0	1
$*\{q_1, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_2, q_3\}$	$\{q_3\}$	$\{q_3\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$*\{q_1, q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$
$*\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\emptyset$	$\emptyset$	$\emptyset$

构造与ε-NFA等价的DFA

- 例:将下图L的ε-NFA，转为等价的DFA。
  - ε-NFA到DFA



	0	1
→$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$
*$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
*$\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
*$\{q_0, q_3\}$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$



小结

- 有穷自动机基本概念
- 三种识别正则语言的有穷自动机
  - 确定有穷自动机（DFA）
  - 非确定有穷自动机（NFA）
  - 带有空转移的有穷自动机（ε-NFA）
- 构造与NFA和ε-NFA等价的DFA的方法
  - 子集构造法